

Shortest Paths Algorithms

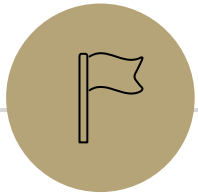
Lecture 5.1, Spring 2026

BFS

Dijkstra's Algorithm

Bellman-Ford Algorithm

BFS

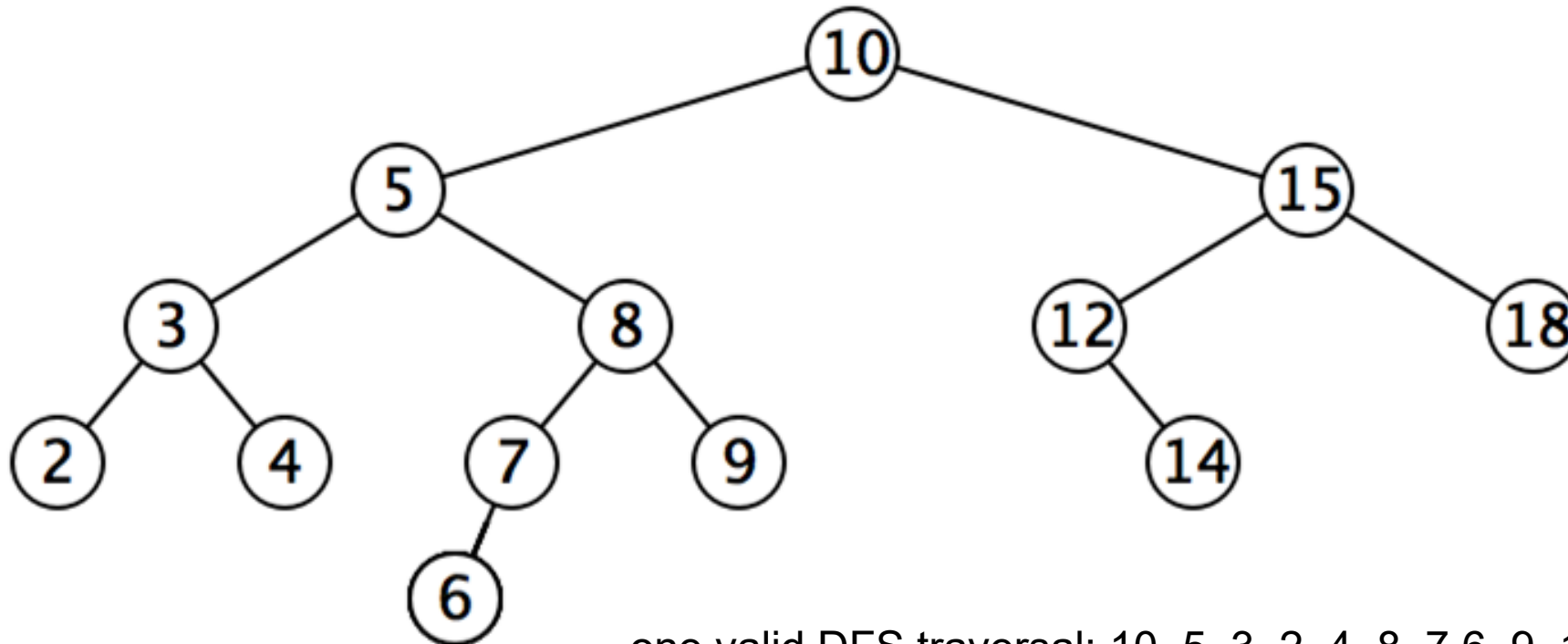


Dijkstra's Algorithm

Bellman-Ford Algorithm

Review: Graph traversal w/ DFS

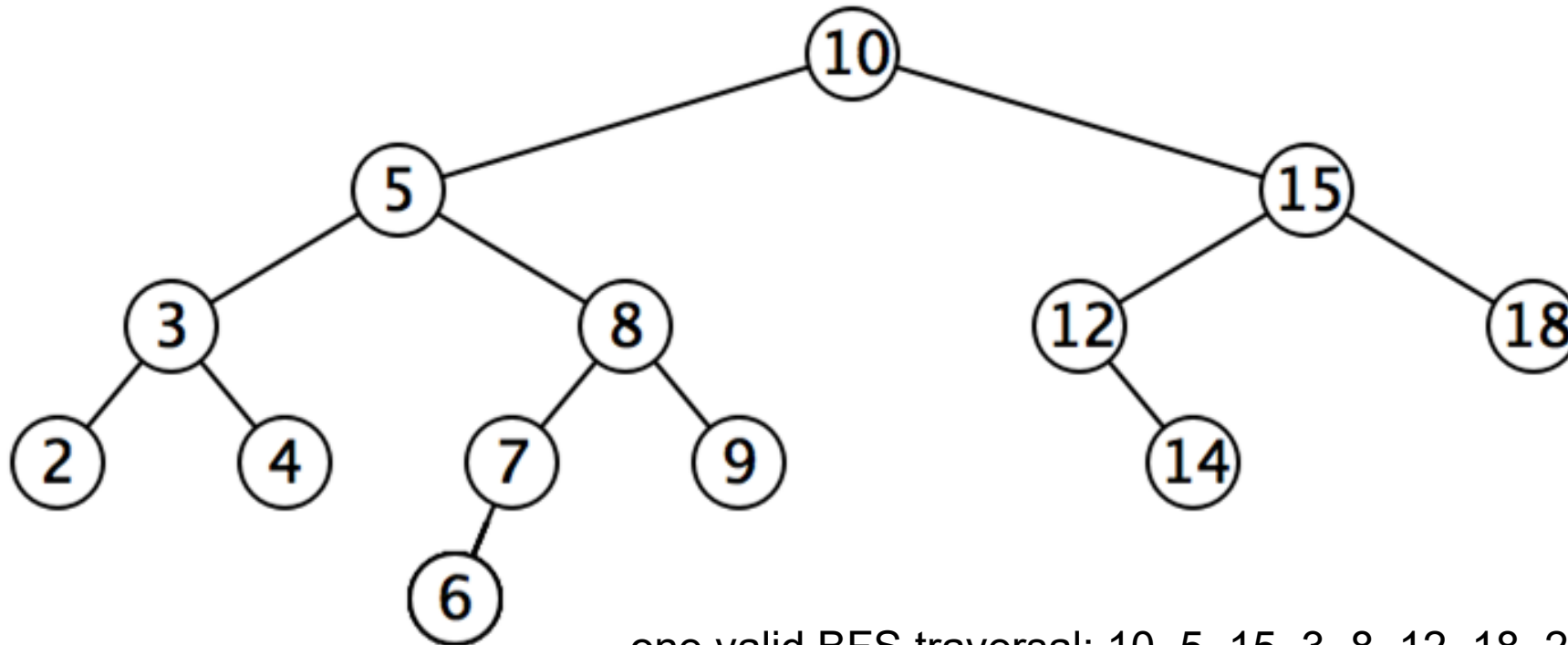
- **Depth** First Search: go as far as you can down one path till you hit a dead end (no neighbors, or no unvisited neighbors). Once you hit a dead end, backtrack and try other edges that you have not tried yet
- Analogy of wandering a maze – if you get stuck at a dead end, trace your steps backwards to the previous fork in the road, and try a different path



one valid DFS traversal: 10, 5, 3, 2, 4, 8, 7, 6, 9, 15, 12, 14, 18

Review: Graph traversal w/ BFS

- **Breadth** First Search - traverse level by level, and visit 1-hop neighbors before 2-hop neighbors before 3-hop neighbors...
- Analogy: sound wave spreading from a starting point, going outwards in all directions; mold on a piece of food spreading outwards so that it eventually covers the whole surface



one valid BFS traversal: 10, 5, 15, 3, 8, 12, 18, 2, 4, 7, 9, 14, 6

BFS for (Unweighted) Shortest Path Problem

- BFS can find shortest paths in an unweighted graph
 - BFS visits nodes in order of their distance from the source node, ensuring the first path found to any node is the shortest possible path in terms of the number of edges
 - Time complexity: $O(V+E)$
- Advantages:
 - Optimal for unweighted graphs
 - Simple implementation
- Limitations:
 - Only works for unweighted graphs

(Unweighted) Shortest Path Problem

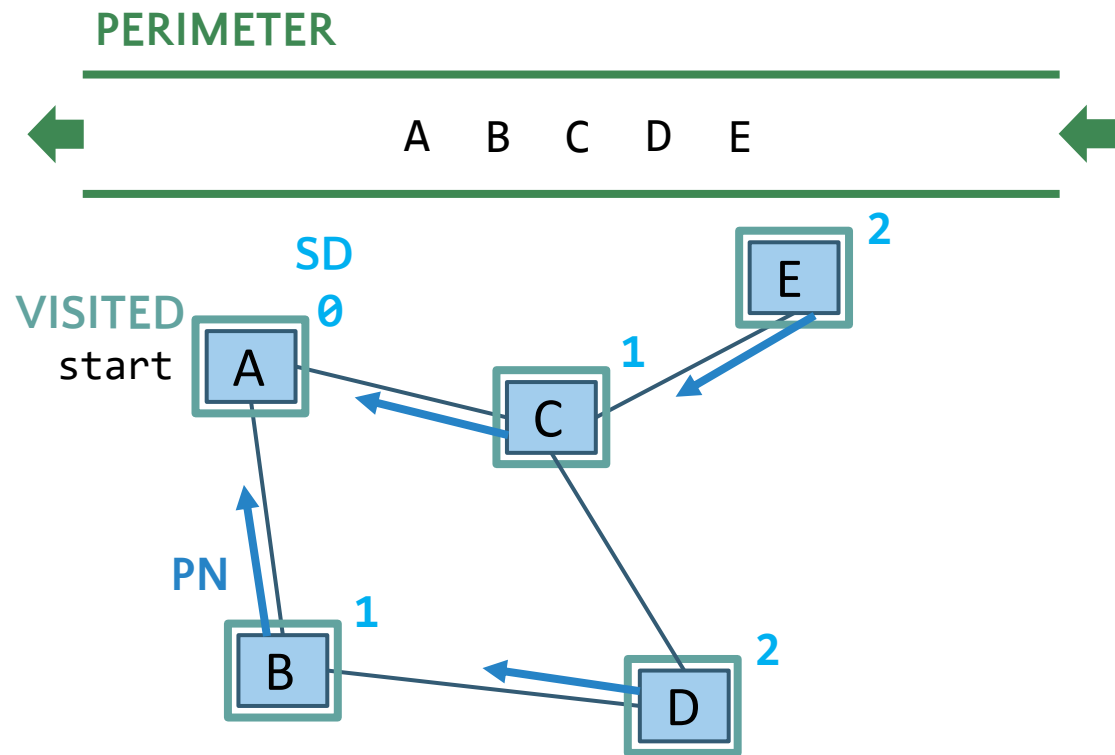
Given source node s (start) and a target node t , how long is the shortest path from s to t ? What edges makeup that path?

BFS for Shortest Paths in an Unweighted Graph

Keep track of how far each node is from the start with two maps

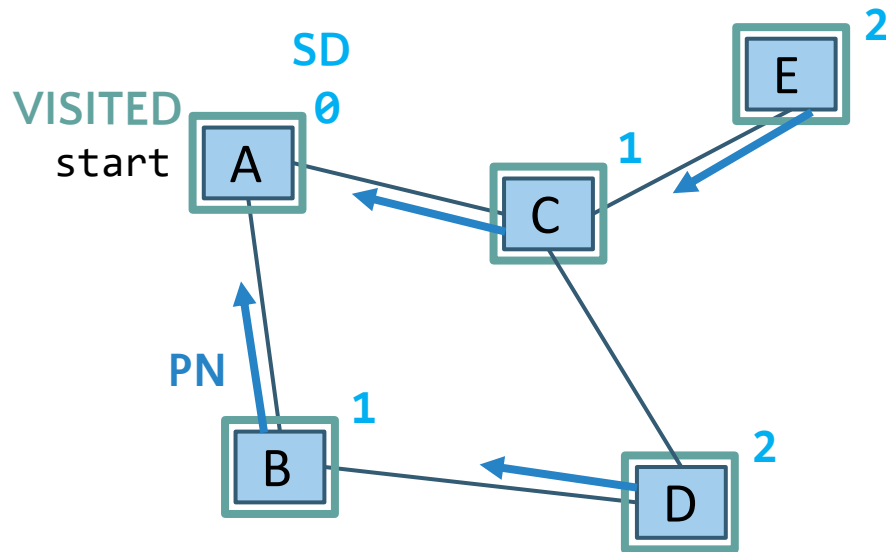
SD: Shortest Distance from source node

PN: Previous Node stores backpointers: each node remembers what node was used to arrive at it



```
...  
Map<Node, Edge> PN = ...  
Map<Node, Double> SD = ...  
  
PN.put(start, null);  
SD.put(start, 0.0);  
  
while (!perimeter.isEmpty()) {  
    Node from = perimeter.remove();  
    for (Edge edge : graph.edgesFrom(from)) {  
        Node to = edge.to();  
        if (!visited.contains(to)) {  
            PN.put(to, edge);  
            SD.put(to, SD.get(from) + 1);  
            perimeter.add(to);  
            visited.add(to);  
        }  
    }  
}  
return PN;  
}
```

Shortest Path Tree



Node	SD	PN
A	0	/
B	1	A
C	1	A
D	2	B or C
E	2	C

The table of SD/PN encodes the Shortest Path Tree (SPT), which encodes the shortest path and distance from the start node to *every other node*

Shortest path to any node can be obtained from SPT

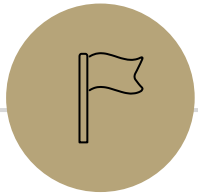
- Length of shortest path from A to D?
 - Lookup in SD map: 2
- What's the shortest path from A to D?
 - Build the path backwards from PN: start at D, follow backpointer to B, follow backpointer to A – the shortest path is ABD

Depending on the order of visiting A's successors with BFS: either B before C, or C before B, D's PN may be either B or C

BFS Time Complexity

- Using Adjacency List: $O(V + E)$
 - Each node is processed exactly once: $O(V)$
 - Each edge is examined exactly once: $O(E)$
 - Total complexity: $O(V + E)$
 - Efficient for sparse graphs (where E is much less than V^2)
- Using Adjacency Matrix: $O(V^2)$
 - For each node, we must check all possible edges to other vertices
 - This results in $O(V^2)$ operations regardless of the actual number of edges

BFS

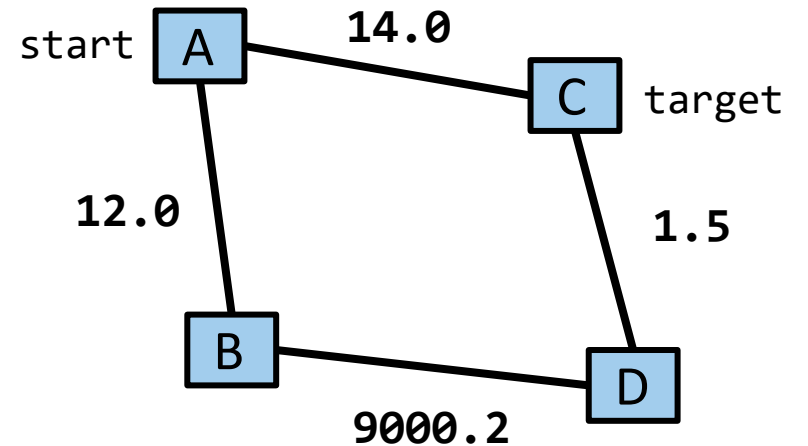


Dijkstra's Algorithm

Bellman-Ford Algorithm

Dijkstra's Algorithm

- Named after its inventor, Edsger W. Dijkstra (1930-2002)
 - 1972 Turing Award
- Solves the Shortest Path Problem on a weighted graph

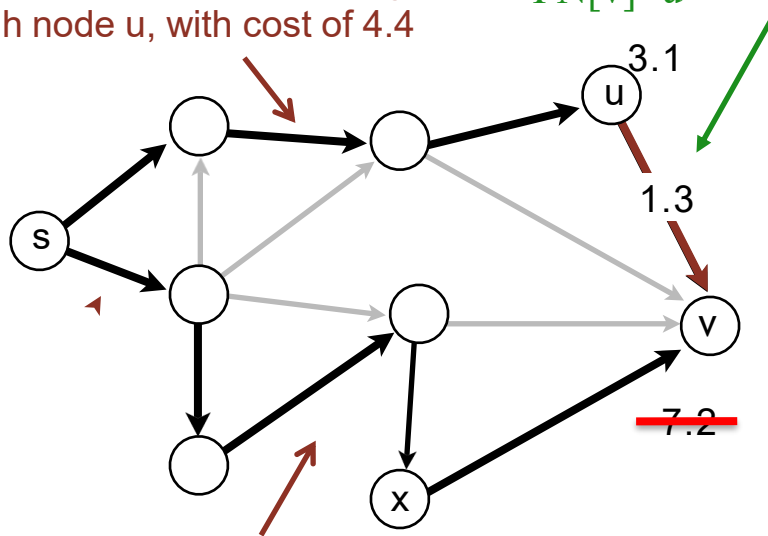


Edge Relaxation

Relax edge $e = u \rightarrow v$ with weight $w(u,v)$. (We also write edge uv to denote $u \rightarrow v$)

- $SD[u]$ is length of shortest known path from s to u .
- $SD[v]$ is length of shortest known path from s to v .
- $PN[v]$ is the previous node on shortest known path from s to v .
- If $e = u \rightarrow v$ gives shorter path to v through u , update $SD[v]$ and $PN[v]$.
 - $SD[v] = \min(SD[v], SD[u] + w(u,v))$; $PN[v]=u$

After relaxing edge uv , the shortest path from s to v is updated to go through node u , with cost of 4.4



Previous shortest path from s to v goes through node x , with cost of 7.2

```
private void relax(DirectedEdge e)
{
    Int u = e.from(), v = e.to();
    if (SD[v] > SD[u] + w(u,v))
    {
        SD[v] = SD[u] + w(u,v);
        PN[v] = u;
    }
}
```

OLD $PN(v)=x$, $SD[v] = 7.2 > SD[u] + w(u,v) = 3.1+1.3 = 4.4$
NEW $SD[v] \leftarrow SD[u] + w(u,v) = 4.4$, $PN[v] = u$

Generic Shortest-paths Algorithm

Generic algorithm (to compute SPT from s)

For each node v : $SD[v] = \infty$.

For each node v : $PN[v] = \text{null}$.

$SD[s] = 0$.

Repeat until done:

- Relax any edge.

Proposition. Generic algorithm computes SPT (if it exists) from s .

Proof.

- Throughout algorithm, $SD[v]$ is the length of a simple path from s to v (and $PN[v]$ is its previous node on the path).
- Each successful relaxation decreases $SD[v]$ for some v .
- The entry $SD[v]$ can decrease at most a finite number of times.

Efficient implementations. How to choose which edge to relax?

- Ex 1. Dijkstra's algorithm. (**no negative weights**)
- Ex 2. Topological sort. (**DAG with no directed cycles**)
- Ex 3. Bellman–Ford algorithm. (**negative weights, can detect negative cycles**)

Dijkstra's Algorithm

- Initialization:

- Set the distance to the source node as 0 and to all other nodes as infinity.
- Mark all nodes as unvisited and store them in a priority queue.

- Main Loop:

- Visit the **unvisited node u** with **the shortest known distance (minimum SD)** from the queue.
- For each **unvisited neighbor node v of node u** , calculate its tentative distance through the current node. **If this distance is smaller than the previously recorded distance, update it with edge relaxation for edge uv .**
- Mark the current node as visited once all its neighbors are processed.

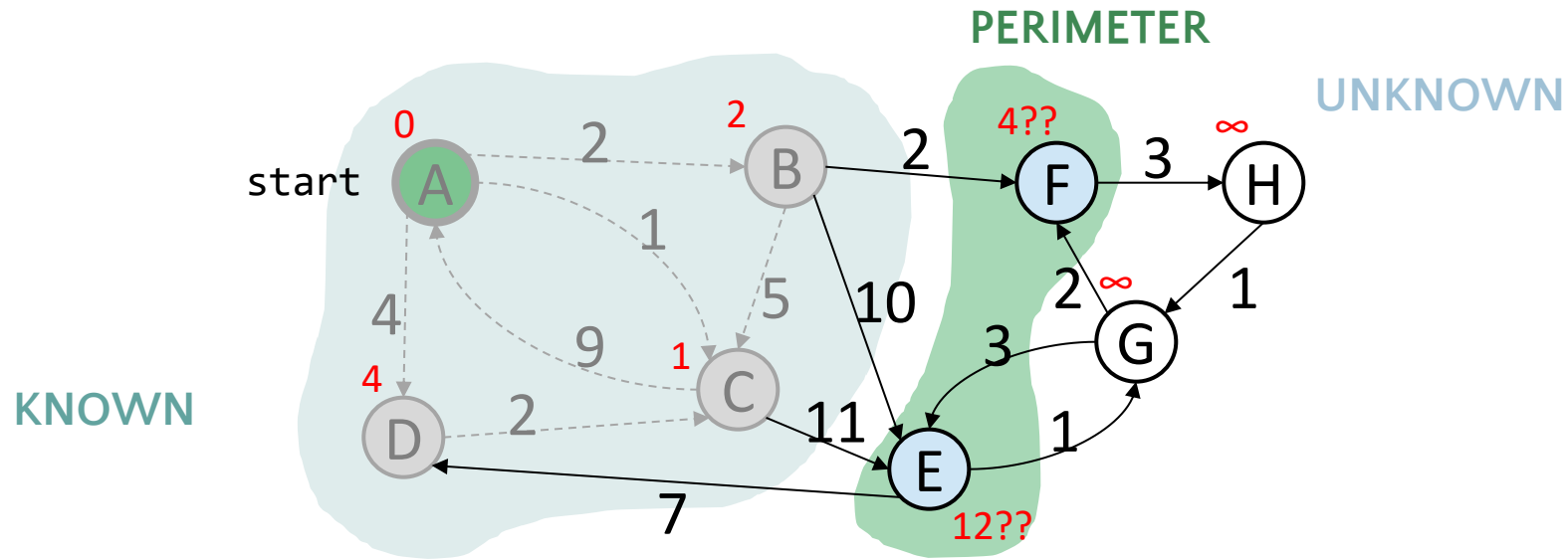
- Termination:

- The algorithm continues until all reachable nodes are visited.

- Notes:

- Greedy and optimal algorithm: any node that has been visited should have its shortest distance to the source.
- It works for both undirected and directed graphs. The only difference is how to get neighbors of node v , as each undirected edge is treated as two directed edges in both directions.

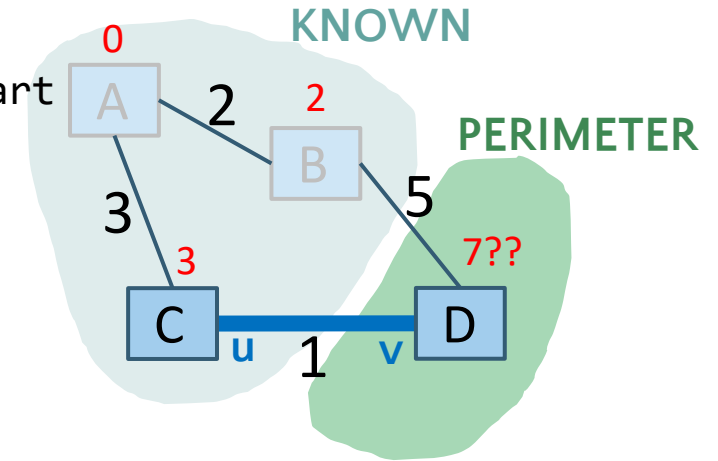
Dijkstra's Algorithm: Idea



- **Initialization:**
 - Start node has distance **0**; all other nodes have distance **∞**
- **At each step:**
 - Pick the closest unknown node v (with smallest SD)
 - Add it to the “cloud” of known nodes (set of nodes whose shortest distance has been computed)
 - Update “best-so-far” distances for nodes with edges from v

Dijkstra's Pseudocode (High-Level)

- Suppose we already visited B, $SD[D] = 7$
- Now considering edge (C, D):
 - $oldDist = 7$
 - $newDist = 3 + 1$
 - Relaxation updates $SD[D]$, $PN[D]$



Similar to “visited” in BFS, “known” is set of nodes that have been visited and we know shortest paths to them

Init all paths to infinite.

Greedy algo: visit closest node first

Consider all nodes reachable from the newly-added node u : would getting there *through* u be a shorter path than their current path length?

```
dijkstraShortestPath(G graph, V start)
```

```
Set known; Map PN, SD;
```

```
initialize SD with all nodes mapped to  $\infty$ , except start to 0
```

```
while (there are unknown nodes):
```

```
let  $u$  be the closest unknown node
```

```
known.add( $u$ );
```

```
for each edge ( $u, v$ ) from  $u$  with weight  $w$ :
```

```
oldDist = SD.get( $v$ ) // previous best path to  $v$ 
```

```
newDist = SD.get( $u$ ) +  $w$  // what if we went through  $u$ ?
```

```
if (newDist < oldDist):
```

```
SD.put( $v$ , newDist)
```

```
PN.put( $v$ ,  $u$ )
```

Dijkstra's Algorithm: Key Properties

Once a node is visited (marked known), its shortest path is known. Can reconstruct path by following back-pointers (in PN map)

While a node is not yet visited/known, another shorter path might be found. We call this update **relaxing** the distance because it only ever shortens the current best path

If we only need path to a specific node, can stop early once that node is visited, and return a partial shortest path tree

```
dijkstraShortestPath(G graph, V start)
  Set known; Map PN, SD;
  initialize SD with all nodes mapped to  $\infty$ , except start to 0

  while (there are unknown nodes):
    let u be the closest unknown node
    known.add(u)
    for each edge (u,v) to unknown v with weight w:
      oldDist = SD.get(v)           // previous best path to v
      newDist = SD.get(u) + w       // what if we went through u?
      if (newDist < oldDist):
        SD.put(v, newDist)
        PN.put(v, u)
```


Dijkstra's Algorithm: Runtime

$O(V)$

$O(V)$

$O(\log V)$ using binary min-heap implementation of a priority queue

$O(E)$

$O(\log V)$

Initialization: $O(V)$
Extracting nodes: $O(V \log V)$
Edge relaxations: $O(E \log V)$
Total runtime: $O((V+E) \log V)$

```
dijkstraShortestPath(G graph, V start)
  Set known; Map PN, SD;
  initialize SD with all nodes mapped to  $\infty$ , except start to 0

  while (there are unknown nodes):
    let u be the closest unknown node
    known.add(u)
    for each edge (u,v) to unknown v with weight w:
      oldDist = SD.get(v)           // previous best path to v
      newDist = SD.get(u) + w       // what if we went through u?
      if (newDist < oldDist):
        SD.put(v, newDist)
        PN.put(v, u)
    update distance in list of unknown nodes
```

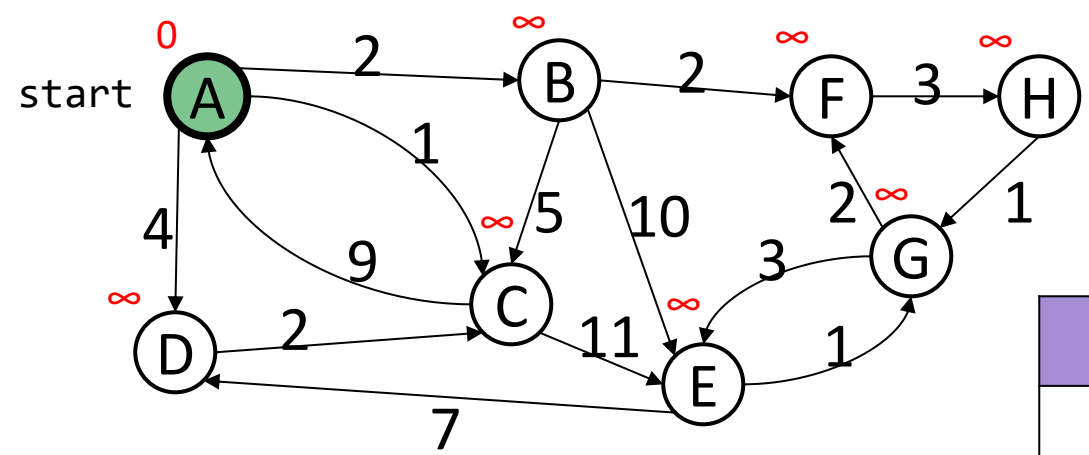
Greedy Algorithms

- A greedy algorithm makes the locally optimal choice at each step
- Dijkstra's is "greedy" because once a node is marked as visited, it is never revisited
 - This is why Dijkstra's does not work with negative edge weights
- In the lecture and exams, when there are multiple possible orders of visiting the next node (with equal SD value), select the next node in alphabetical or numerical order
 - The intermediate steps will depend on the order, but final result will be the same
- Other examples of greedy algorithms are:
 - Kruskal and Prim's minimum spanning tree algorithms

Resolving Ambiguities

- As There are typically multiple possible orders of the same graph. In the lecture and exams, we often use the following rule to resolve any ambiguities:
- “When there are multiple possible orders of visiting the next node, select the next node in alphabetical or numerical order.”

Example I

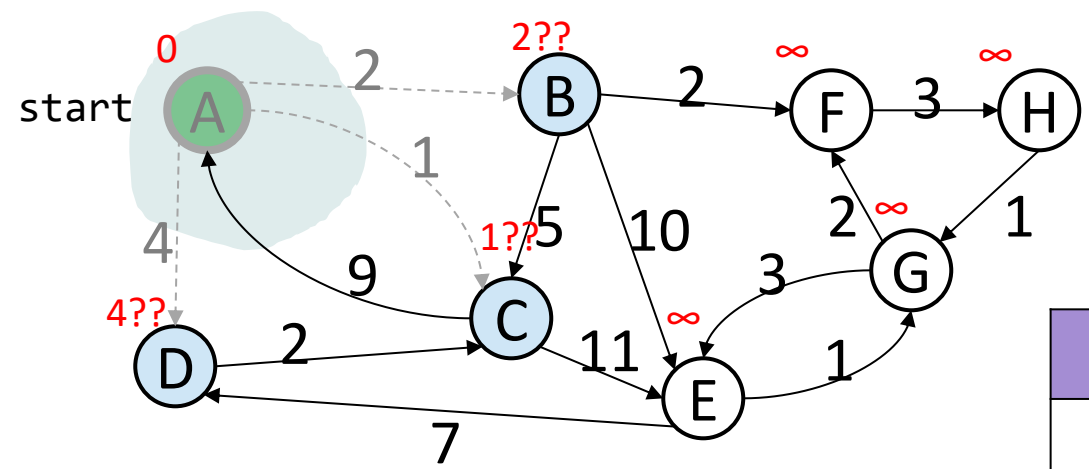


Visit Order

Start from the source node A

Node	SD	PN
A	0	/
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	
G	∞	
H	∞	

Example I



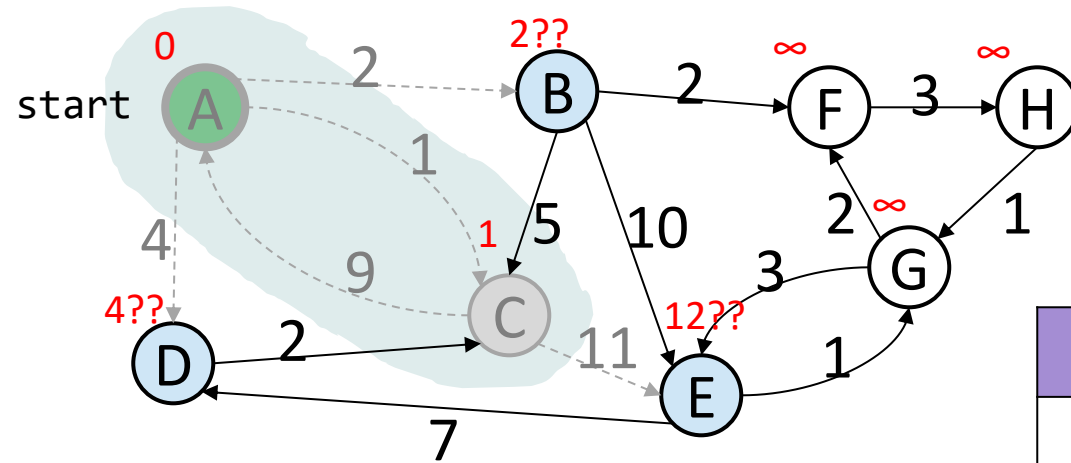
?? Means that SDs have not yet been finalized, as a shortcut may be found in the future.

Visit Order
A

Visit C next, since C has the smallest SD of 1 among all unknown (unvisited) nodes

Node	SD	PN
A	0	/
B	2	A
C	1	A
D	4	A
E	∞	
F	∞	
G	∞	
H	∞	

Example I



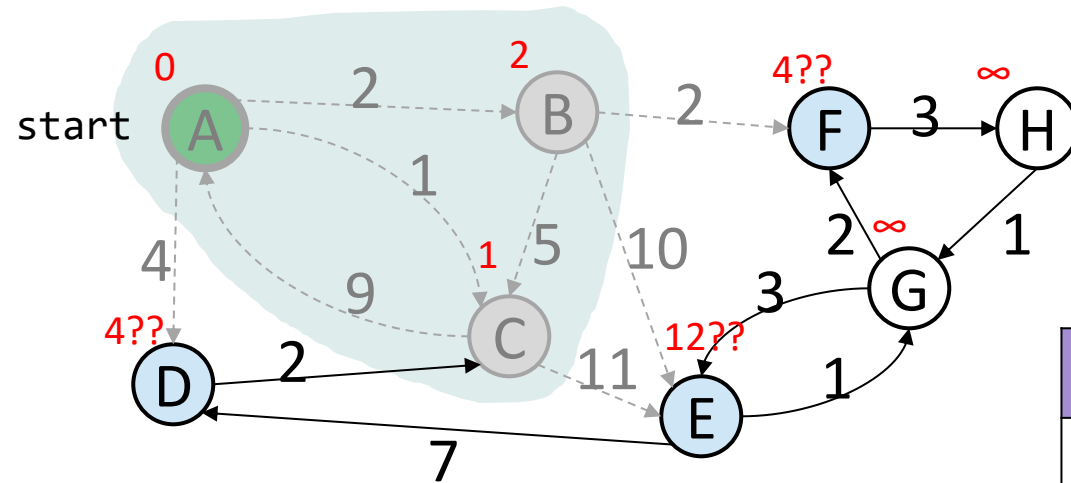
Visit Order

A, C

Visit B next, since B has the smallest SD of 2 among all unvisited nodes

Node	SD	PN
A	0	/
B	2	A
C	1	A
D	4	A
E	12	C
F	∞	
G	∞	
H	∞	

Example I



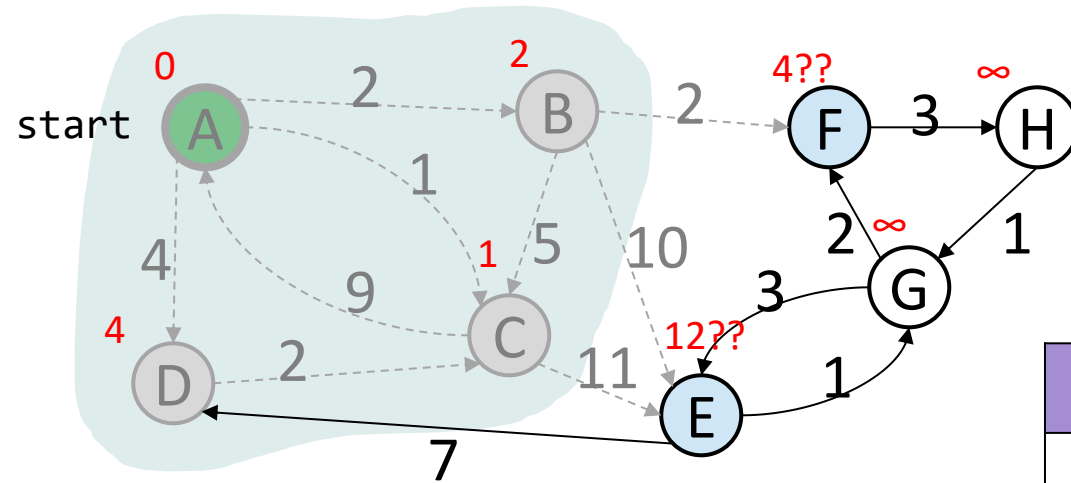
Visit Order

A, C, B

We can choose to visit either D or F next, since they have equal smallest SD of 4 among all unvisited nodes. Let's visit D in alphabetical order

Node	SD	PN
A	0	/
B	2	A
C	1	A
D	4	A
E	12	C
F	4	B
G	∞	
H	∞	

Example I



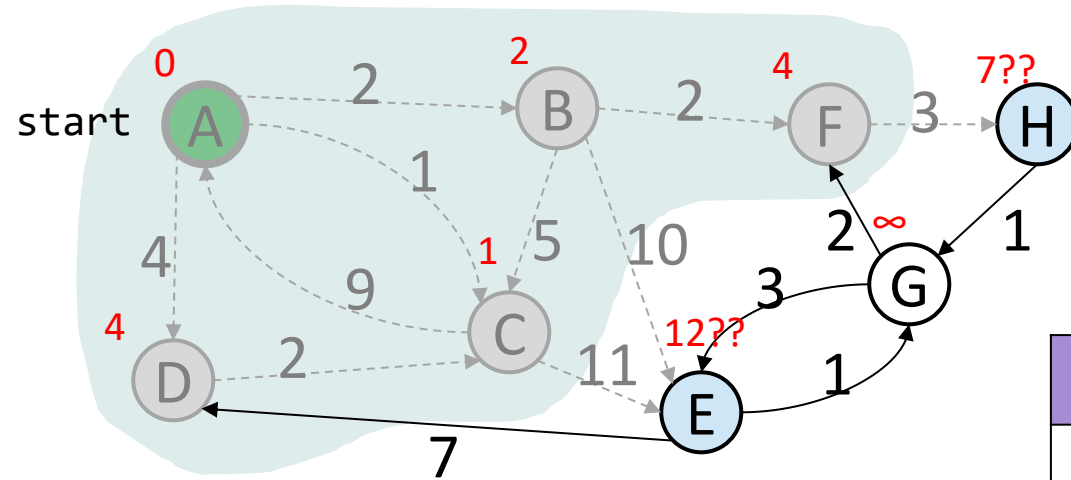
Visit Order

A, C, B, D

Visit F next, since F has the smallest SD of 4 among all unvisited nodes

Node	SD	PN
A	0	/
B	2	A
C	1	A
D	4	A
E	12	C
F	4	B
G	∞	
H	∞	

Example I



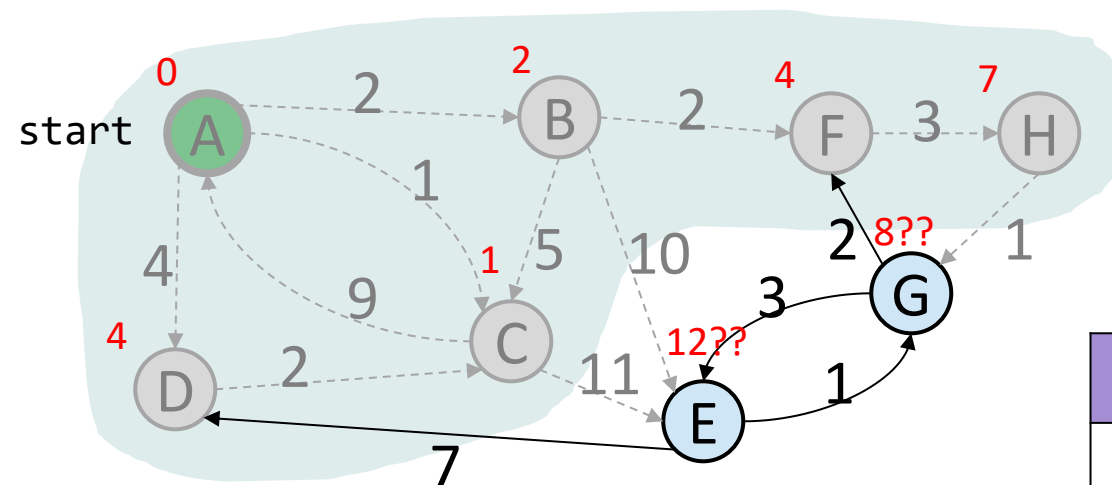
Visit Order

A, C, B, D, F

Visit H next, since H has the smallest SD of 7 among all unvisited nodes

Node	SD	PN
A	0	/
B	2	A
C	1	A
D	4	A
E	12	C
F	4	B
G	∞	
H	7	F

Example I



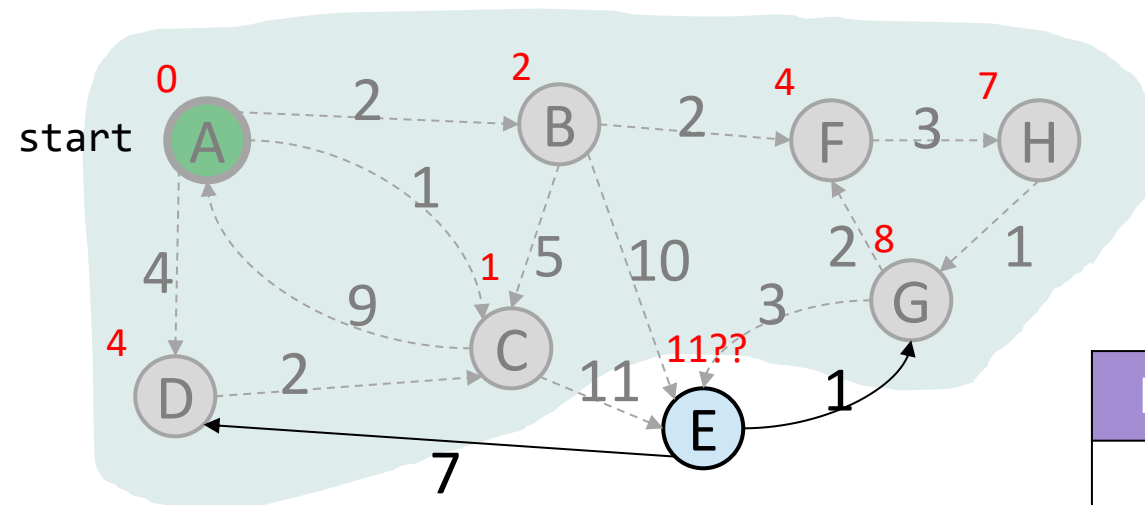
Visit Order

A, C, B, D, F, H

Visit G next, since G has the smallest SD of 8 among all unvisited nodes

Node	SD	PN
A	0	/
B	2	A
C	1	A
D	4	A
E	12	C
F	4	B
G	8	H
H	7	F

Example I



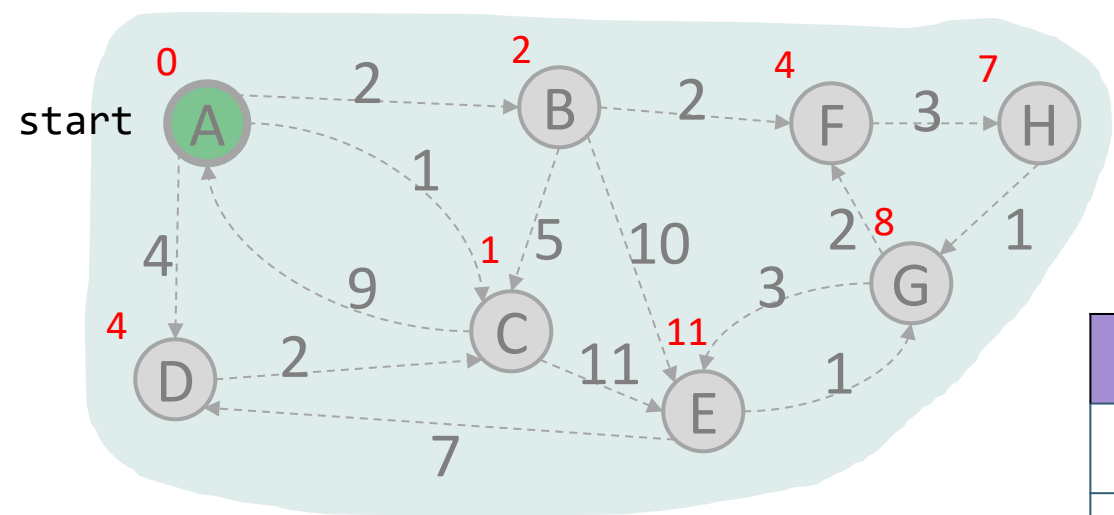
Visit Order

A, C, B, D, F, H, G

We found a shortcut to E going through G, so we update SD and PN for E. Visit E next, since it is the last unvisited node

Node	SD	PN
A	0	/
B	2	A
C	1	A
D	4	A
E	12 11	∈ G
F	4	B
G	8	H
H	7	F

Example I Final

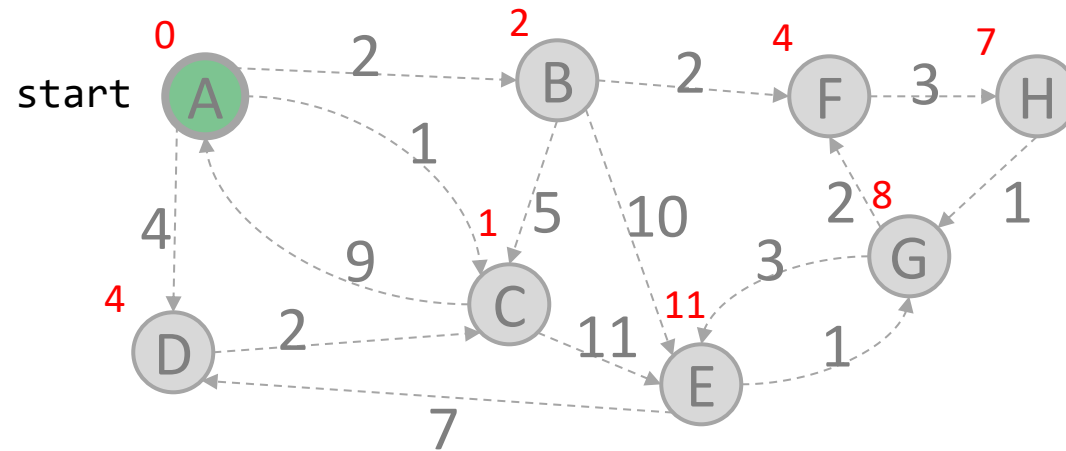


Visit Order
A, C, B, D, F, H, G, E

All nodes have now been visited and are known

Node	SD	PN
A	0	/
B	2	A
C	1	A
D	4	A
E	12 11	∈ G
F	4	B
G	7 8	8 H
H	7	F

Example I: Interpreting the Results



How to get the shortest path from A to E?

- Follow PN backpointers to get path ABFHGE

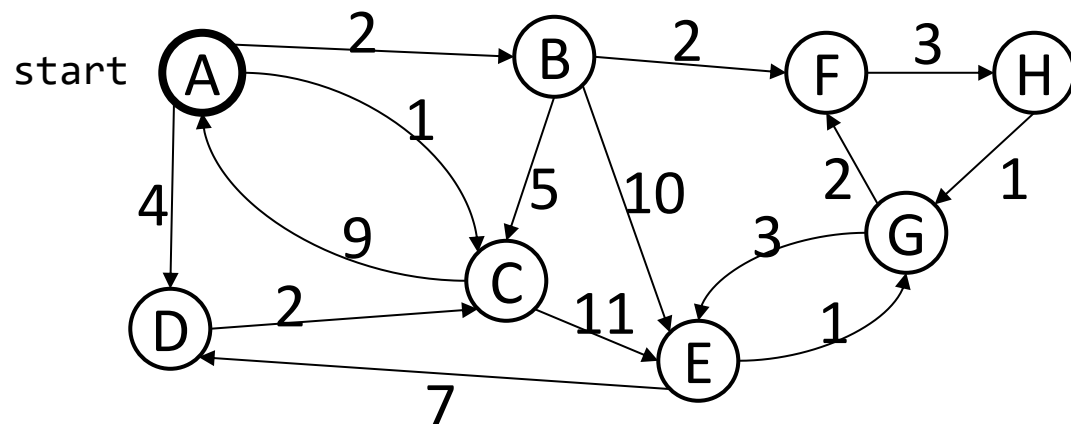
Visit Order

A, C, B, D, F, H, G, E

Node	SD	PN
A	0	/
B	2	A
C	1	A
D	4	A
E	11	C
F	4	B
G	8	H
H	7	F

Example I Sample Exam Question and Answer

Given this directed graph, run Dijkstra's Algo to find shortest paths starting from source node A. Give the node visit order, and fill in this table of SN (Shortest Distance) and PN (Previous Node), crossing out old SD and PN as you find a shortcut path with smaller SD



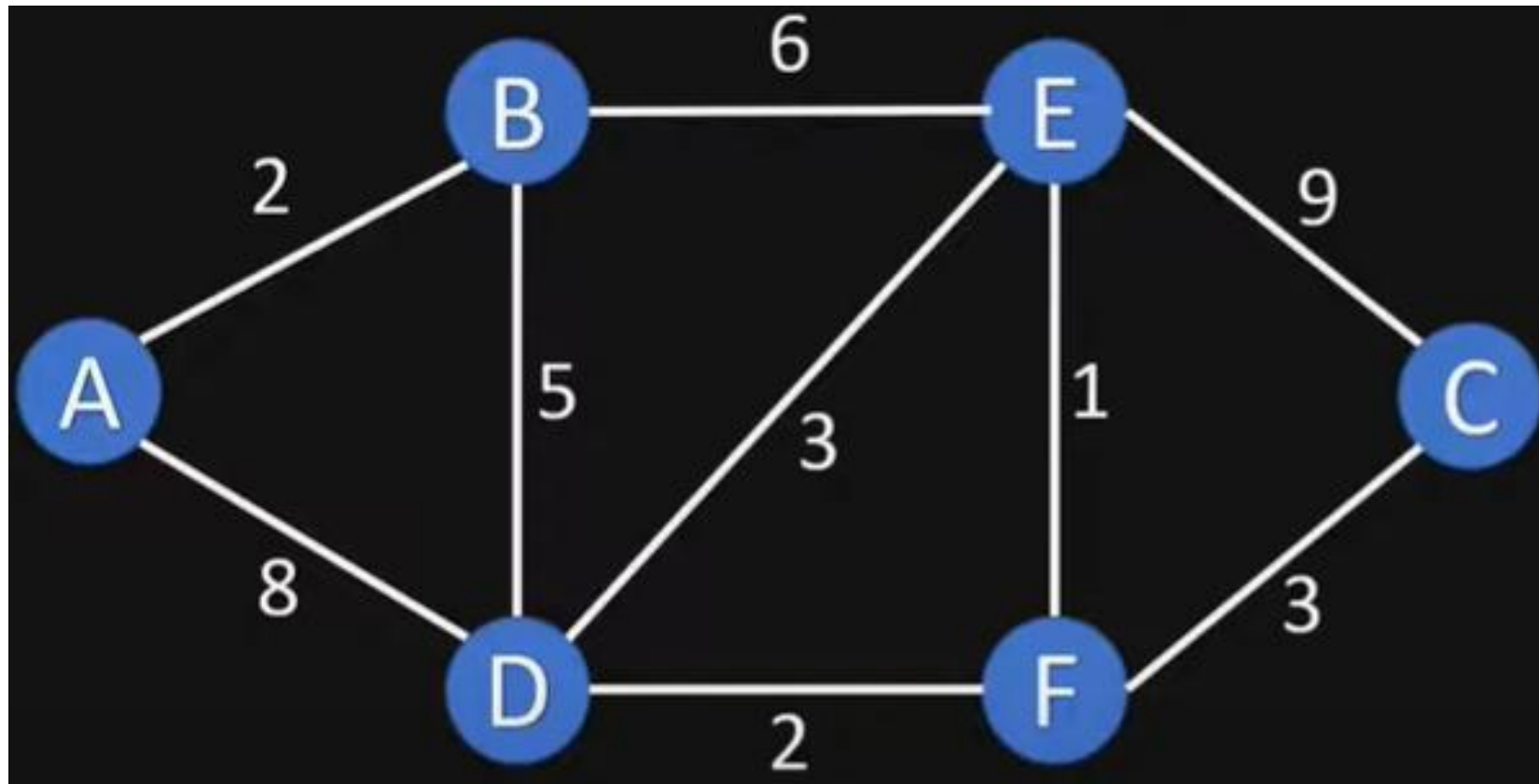
Visit Order

A, C, B, D, F, H, G, E

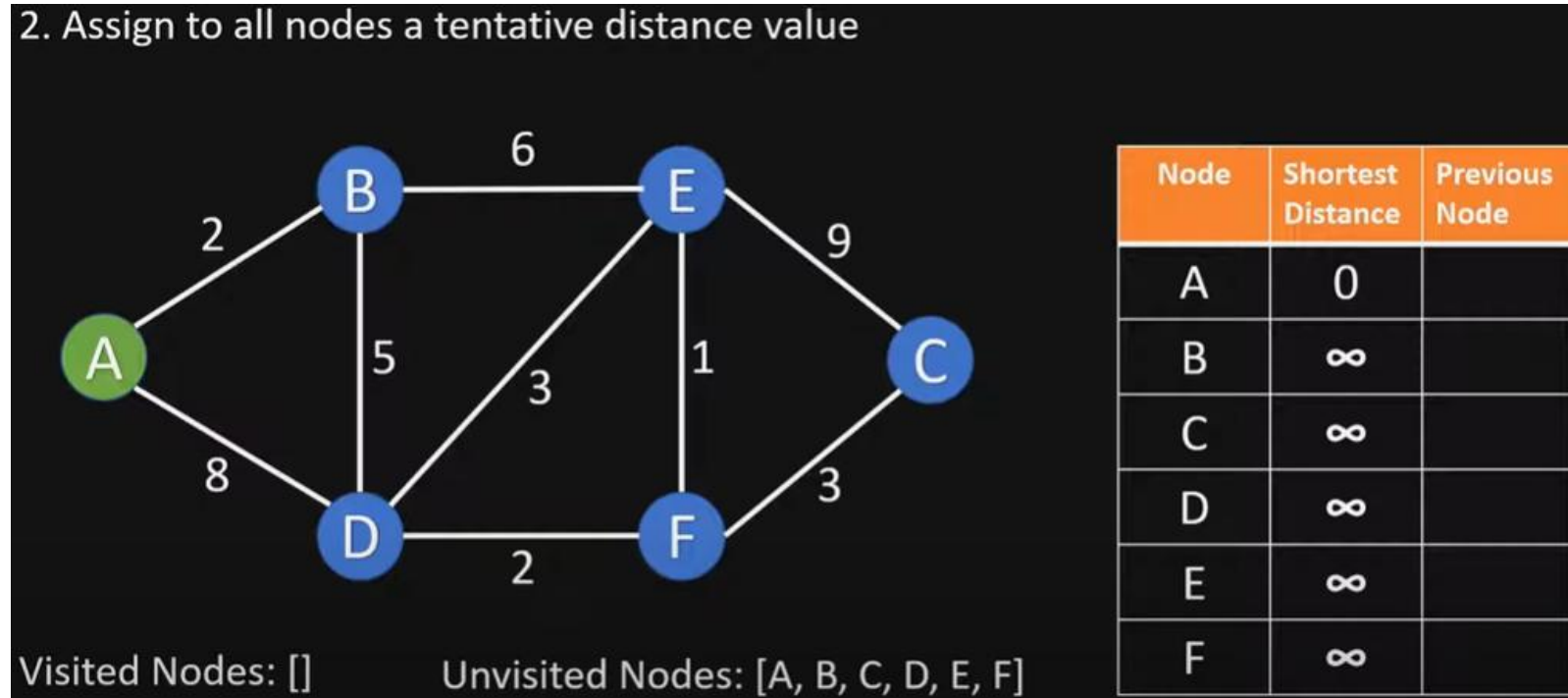
Node	SD	PN
A	0	/
B	2	A
C	1	A
D	4	A
E	12 11	∈ G
F	4	B
G	7 8	8 H
H	7	F

Example II

- Dijkstras Shortest Path Algorithm Explained | With Example | Graph Theory
 - <https://www.youtube.com/watch?v=bZkzH5x0SKU>

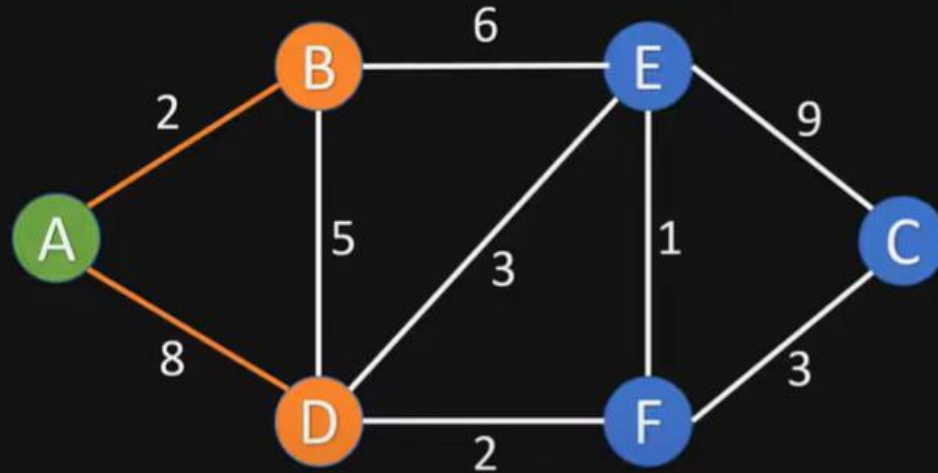


Initialize



Visit node A

3. For the current node calculate the distance to all unvisited neighbours
3.1. Update shortest distance, if new distance is shorter than old distance



Visited Nodes: []

Unvisited Nodes: [A, B, C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

$$\text{OLD SD}[B] = \infty > \text{SD}[A] + w(A,B) = 0+2 = 2$$

$$\text{NEW SD}[B] \leftarrow \text{SD}[A] + w(A,B) = 2, \text{PN}[B] = A$$

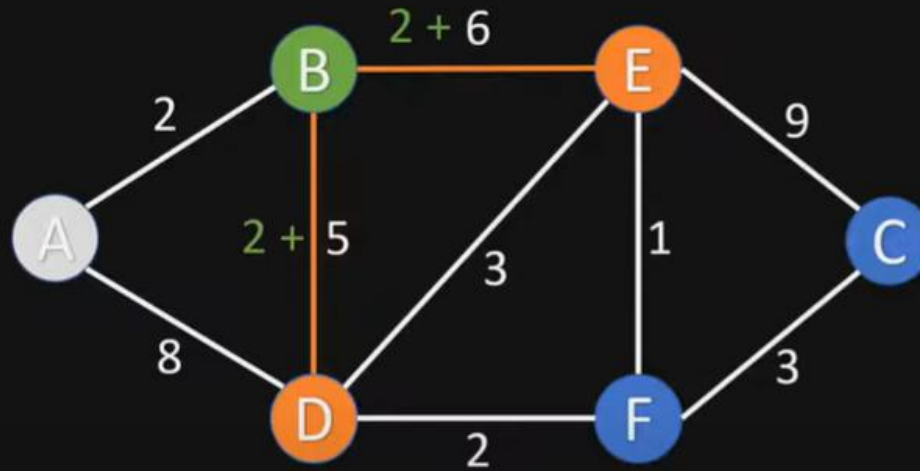
$$\text{OLD SD}[D] = \infty > \text{SD}[A] + w(A,D) = 0+8 = 8$$

$$\text{NEW SD}[D] \leftarrow \text{SD}[A] + w(A,D) = 8, \text{PN}[D] = A$$

Visit node B

3. For the current node calculate the distance to all unvisited neighbours

3.1. Update shortest distance, if new distance is shorter than old distance



Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	∞	

OLD $SD[D] = 8 > SD[B] + w(B,D) = 2+5 = 7$

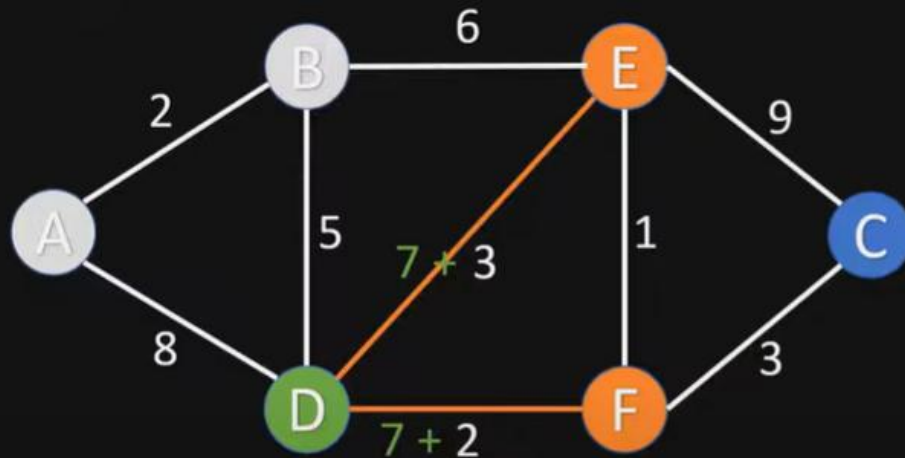
NEW $SD[D] \leftarrow SD[B] + w(B,D) = 7$, $PN[D] = B$

OLD $SD[E] = \infty > SD[B] + w(B,E) = 2+6 = 8$

NEW $SD[E] \leftarrow SD[B] + w(B,E) = 8$, $PN[E] = B$

Visit node D

3. For the current node calculate the distance to all unvisited neighbours
3.1. Update shortest distance, if new distance is shorter than old distance



Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	9	D

OLD $SD[E] = 8 < SD[D] + w(D,E) = 7+3 = 10$

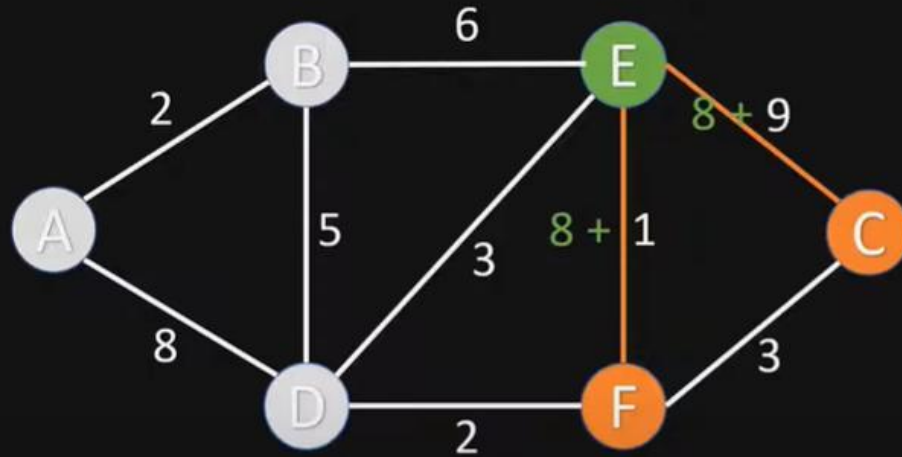
No update, $SD[E]$ stays 8, $PN[E]$ stays B

OLD $SD[F] = \infty > SD[D] + w(D,F) = 7+2 = 9$

NEW $SD[F] \leftarrow SD[D] + w(D,F) = 9$, $PN[F] = D$

Visit node E

3. For the current node calculate the distance to all unvisited neighbours
3.1. Update shortest distance, if new distance is shorter than old distance



Visited Nodes: [A, B, D] Unvisited Nodes: [C, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	17	E
D	7	B
E	8	B
F	9	D

OLD $SD[C] = \infty > SD[E] + w(E.C) = 8 + 9 = 17$

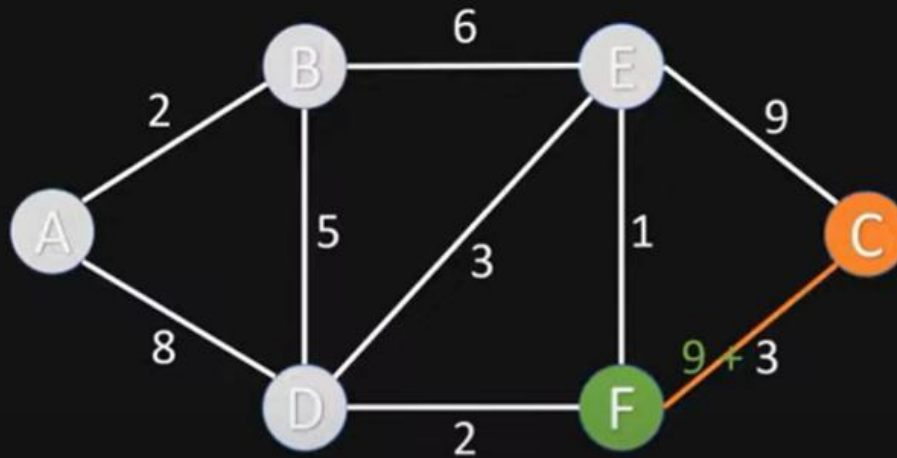
NEW $SD[C] \leftarrow SD[E] + w(E.C) = 17$, $PN[C] = E$

OLD $SD[F] = 9 = SD[E] + w(E.F) = 8 + 1 = 9$

No update, $SD[F]$ stays 9, $PN[F] = D$ (You can also update $PN[F] = E$.)

Visit node F

3. For the current node calculate the distance to all unvisited neighbours
3.1. Update shortest distance, if new distance is shorter than old distance



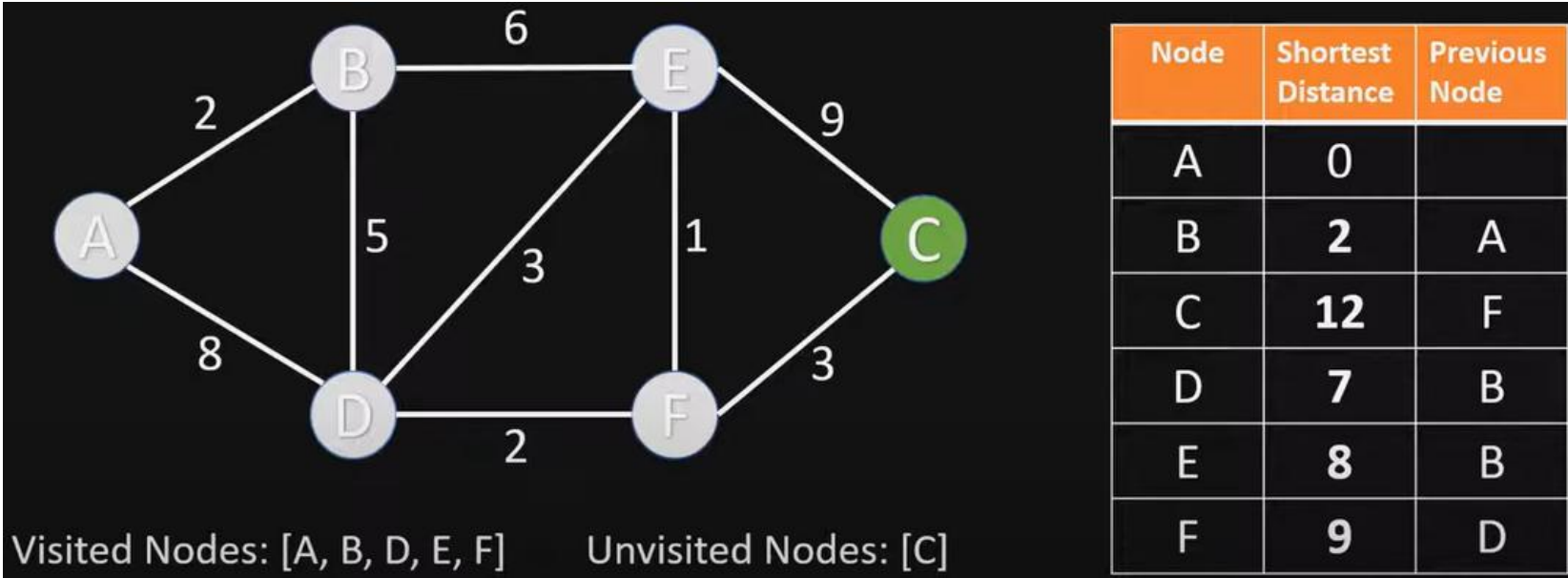
Visited Nodes: [A, B, D, E] Unvisited Nodes: [C, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

$$\text{OLD SD}[C] = 17 > \text{SD}[F] + w(F,C) = 9 + 3 = 12$$

$$\text{NEW SD}[C] \leftarrow \text{SD}[F] + w(F,C) = 12, \text{PN}[C] = F$$

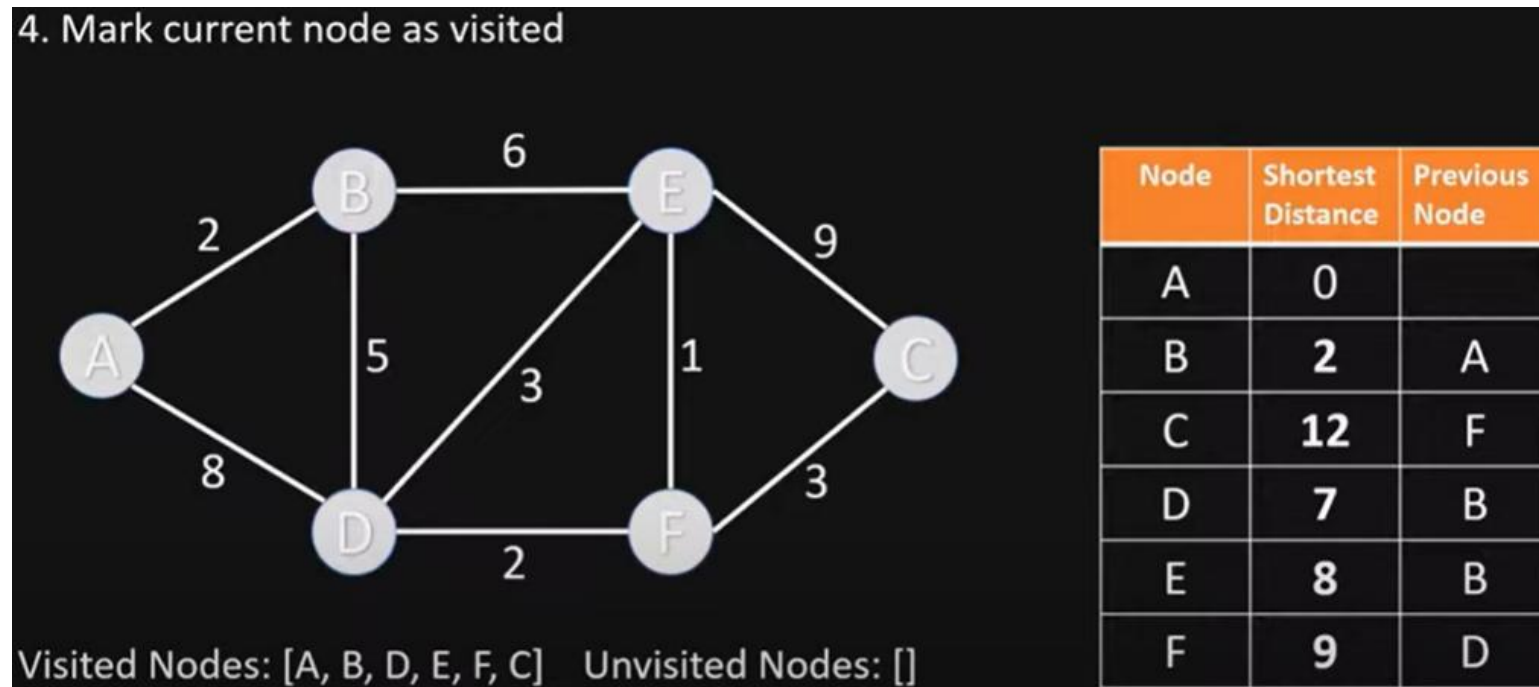
Visit node C



Nothing changes, since C has no unvisited neighbor nodes

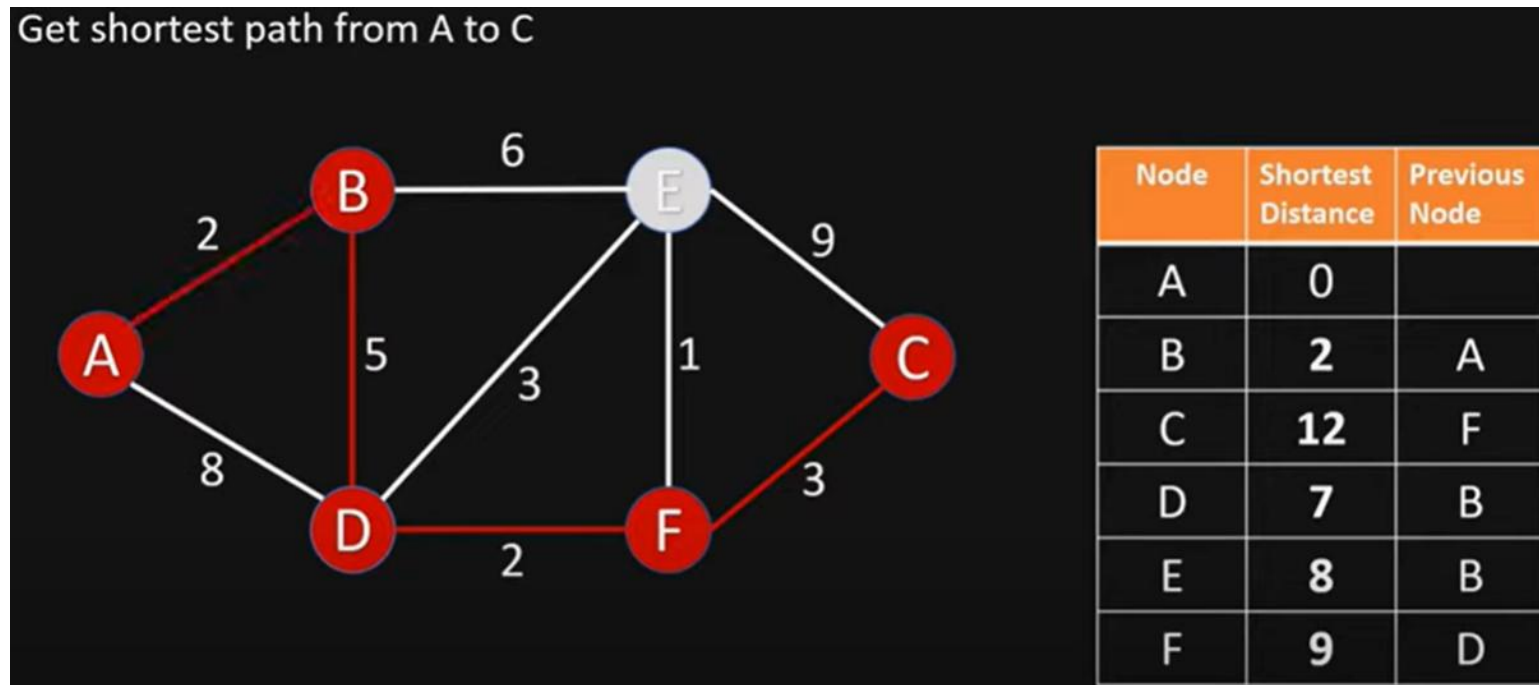
End of Algorithm

- The table now contains the SD (shortest distance) to each node N from the source node A, and its PN (previous node) in the shortest path



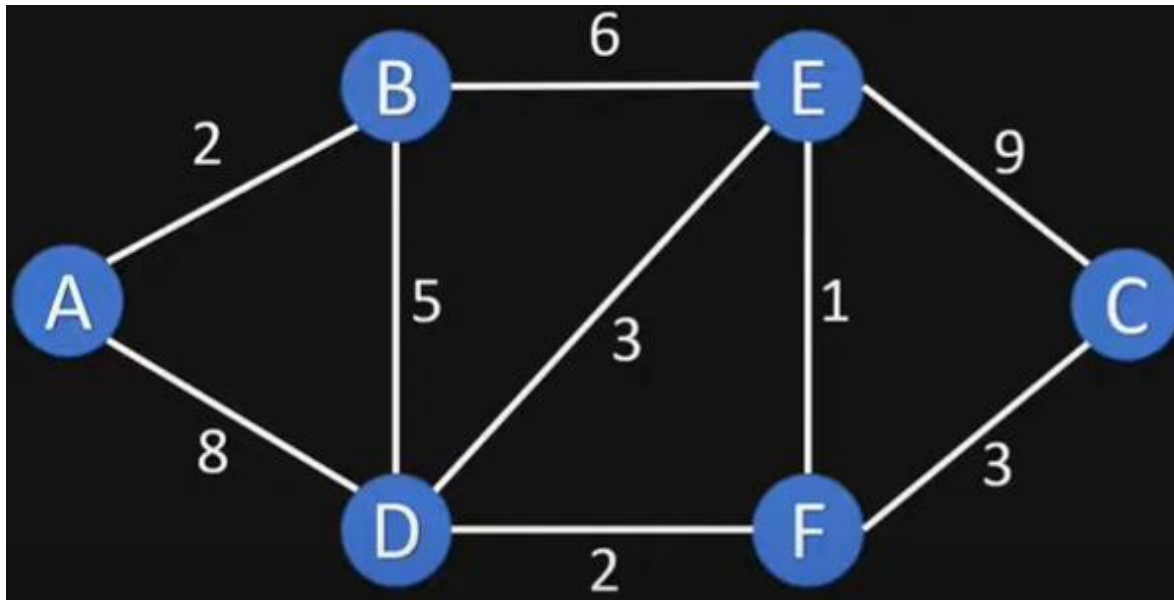
Getting the Shortest Path from A to C

- C's previous node is F; F's previous node is D; D's previous node is B; B's previous node is A
- Shortest Path from A to C is ABDFC



Example II Exam Question and Answer

Given this directed graph, run Dijkstra's Algo to find shortest paths starting from source node A. Give the node visit order, and fill in this table of SN (Shortest Distance) and PN (Previous Node), crossing out old SD and PN as you find a shortcut path with smaller SD



Visit Order

A, B, D, E, F, C

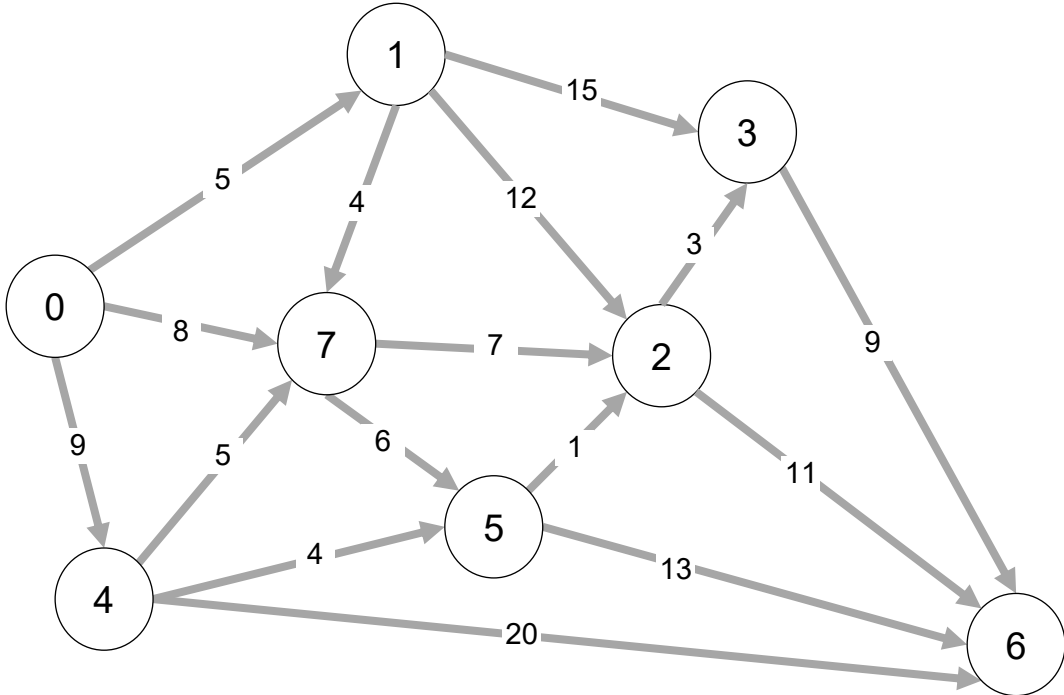
Node	SD	PN
A	0	/
B	2	A
C	17 12	E F
D	8 7	A B
E	8	B
F	9	D

Example III

choose node 5
relax all edges adjacent from 5
choose node 2
relax all edges adjacent from 2
choose node 3
relax all edges adjacent from 3
choose node 6
relax all edges adjacent from 6

Final Answer

Visit Order
0, 1, 7, 4, 5, 2, 3, 6



choose source node 0
relax all edges adjacent from 0
choose node 1
relax all edges adjacent from 1

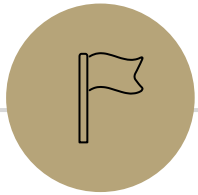
choose node 7
relax all edges adjacent from 7
choose node 4
relax all edges adjacent from 4

v	SD
0	∞ 0
1	∞ 5
2	∞ 17 15 14
3	∞ 20 17
4	∞ 9
5	∞ 14 13
6	∞ 29 26 25
7	∞ 8

v	PN
0	-
1	- 0
2	- 1 7 5
3	- 1 2
4	- 0
5	- 7 4
6	- 4 5 2
7	- 0

Node	SD	PN
0	0	/
1	5	0
2	17 15 14	17 5
3	20 17	1 2
4	9	0
5	14 13	7 4
6	29 26 25	4 5 2
7	8	0

BFS



Dijkstra's Algorithm

Bellman-Ford Algorithm

Bellman-Ford Algorithm

- A shortest path algorithm that works with negative edge weights
- There can be at most $V - 1$ edges in our shortest path
 - If there are V or more edges in a path that means there's a cycle/repeated node
- Run $V - 1$ iterations of shortest path analysis through the graph
 - Repeatedly revisit and update SD and PN
- Look at each vertex's outgoing edges in each iteration
- It is slower than Dijkstra's because it will revisit previously assessed nodes
- Can terminate early when all SD values have converged
- The order of edge relaxations affects algorithm efficiency but not correctness
- Time complexity:
 - Worst Case: $O(VE)$; Average Case: $O(VE)$
- If the graph is dense or complete, the value of E becomes $O(V^2)$. So overall time complexity becomes $O(V^3)$
- Bellman-Ford, by Michael Sambol
 - <https://www.youtube.com/watch?v=9PHkkOUavIM>
 - <https://www.youtube.com/watch?v=obVWXjtGOL64>

Bellman-Ford algorithm

For each node v : $SD[v] = \infty$.

For each node v : $PN[v] = \text{null}$.

$SD[s] = 0$.

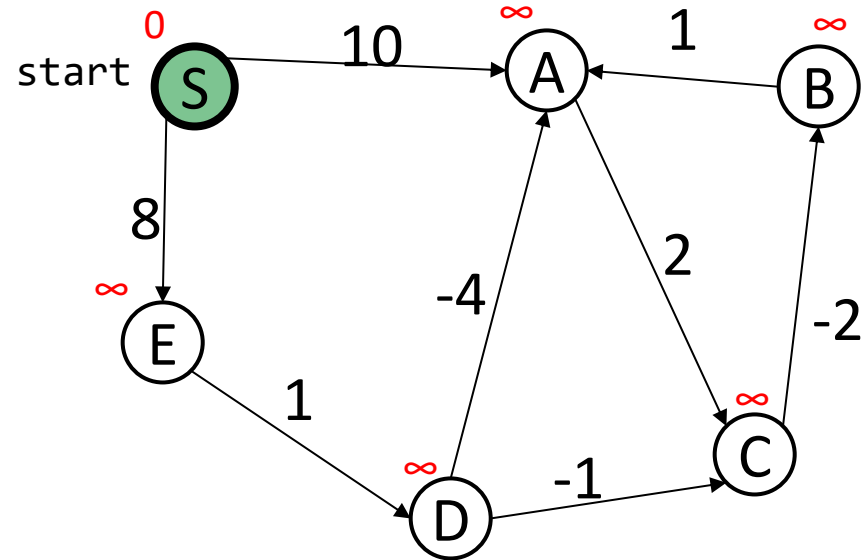
For $i = 1$ to $V-1$:

For each vertex:

Relax all its outgoing edges

In each iteration, we relax every edge once. The order of relaxation may affect how quickly distances converge, but it does not affect correctness, and the final solution is guaranteed to be optimal.

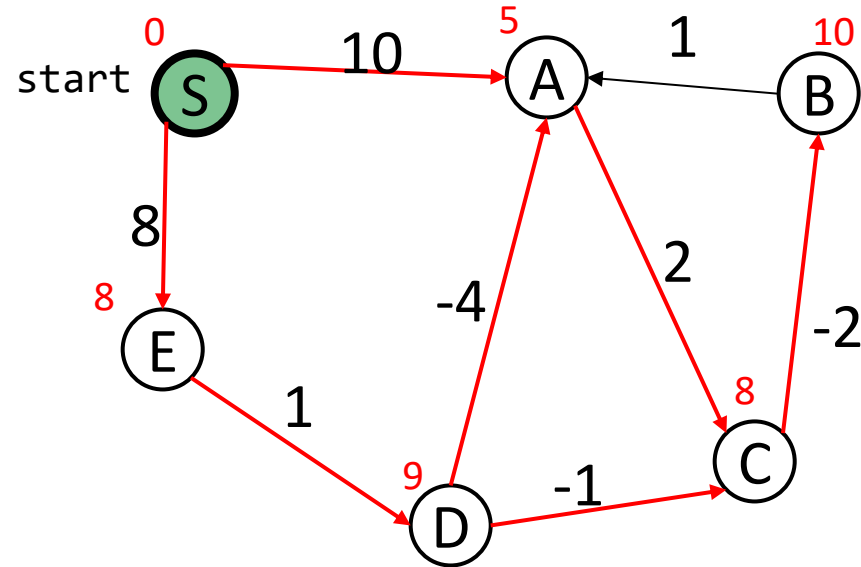
Bellman-Ford Example



node	SD	PN
S	0	
A	∞	
B	∞	
C	∞	
D	∞	
E	∞	

We assume node visit order S, A, C, B, E, D in this example.

Bellman-Ford Example



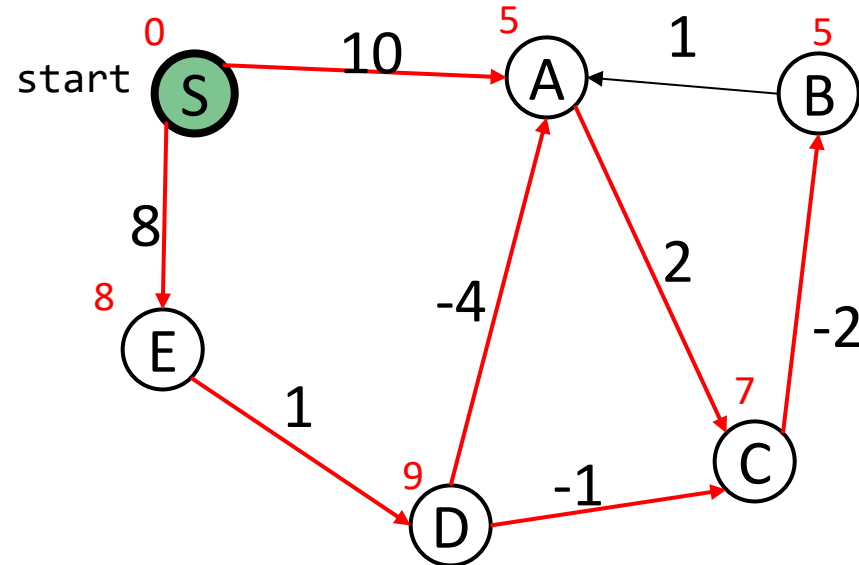
Iteration 1

node	SD	PN
S	0	-
A	5	D
B	10	C
C	8	D
D	9	E
E	8	S

* C's SD was updated to be 12 after visiting A, but it was updated to be 8 after visiting D in the same iteration

We assume node visit order S, A, C, B, E, D in this example.

Bellman-Ford Example



Iteration 2

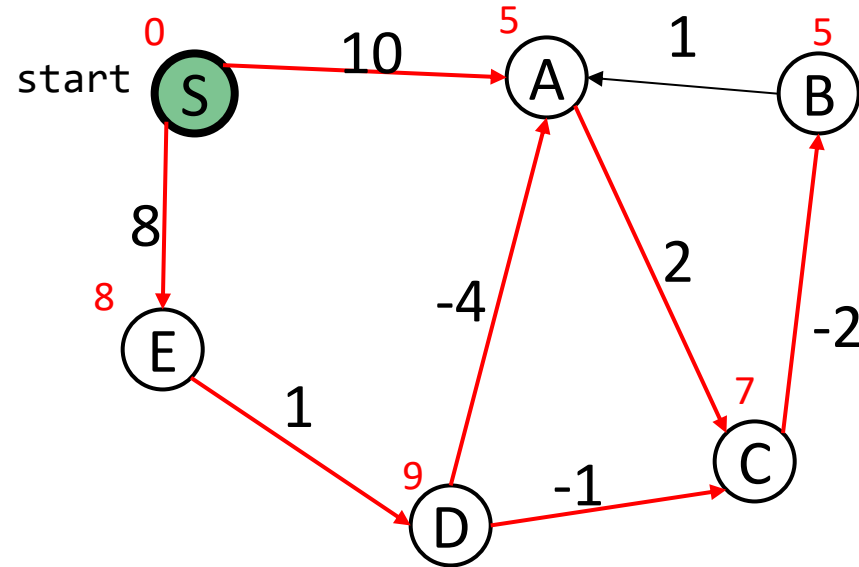
node	SD	PN
S	0	-
A	5	D
B	10 5	C
C	8 7	A
D	9	E
E	8	S

* With a shortened distance to A from iteration 1 we can improve the distance to C

* With a shortened distance to C in this iteration we can improve distance to B

We assume node visit order S, A, C, B, E, D in this example.

Bellman-Ford Example



Iteration 3

node	SD	PN
S	0	-
A	5	D
B	5	C
C	7	A
D	9	E
E	8	S

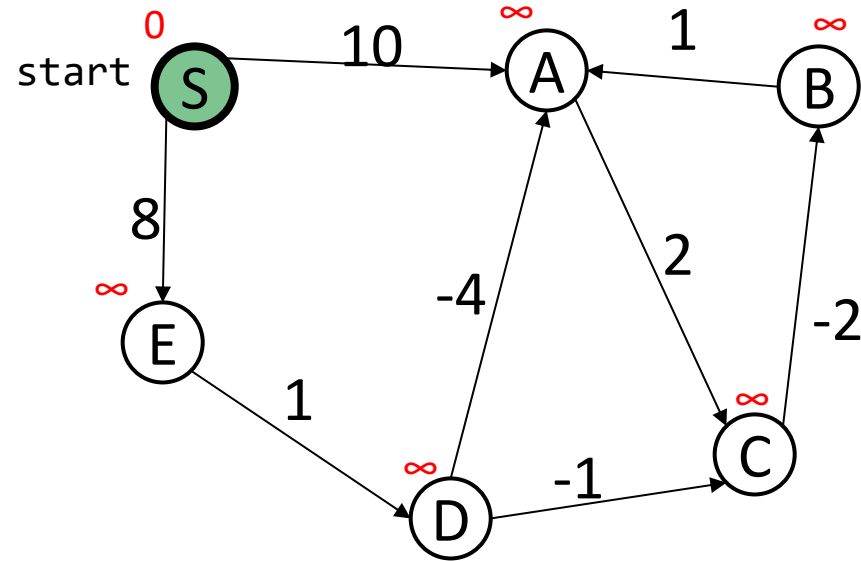
No changes!
This means we can stop early

We assume node visit order S, A, C, B, E, D in this example.

Bellman-Ford Convergence Speed

- Bellman-Ford converged in 3 Iterations (not 5)
- Node Order and Convergence Speed
 - Node order does not affect correctness, but processing vertices in a BFS-like order from the source allows distance improvements to propagate forward within the same iteration, leading to faster convergence.
- Recommendation:
 - Relax vertices in outward (level-by-level) order from the source when possible.
- If we adopt a different node visit order, it does not affect algorithm correctness but may affect convergence speed
 - Let's try node order S, A, C, B, D, E

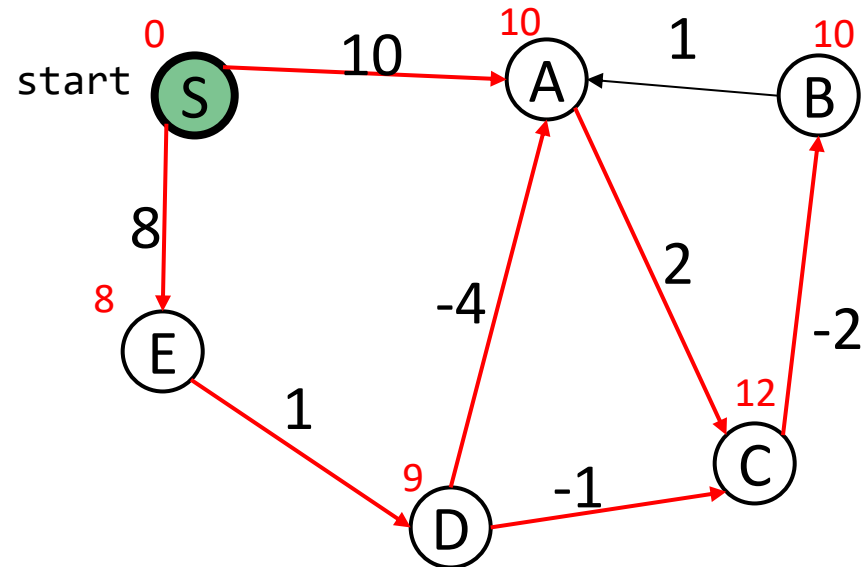
Bellman-Ford Example



node	SD	PN
S	0	
A	∞	
B	∞	
C	∞	
D	∞	
E	∞	

We assume node visit order S, A, C, B, **D**, **E** in this example.

Bellman-Ford Example



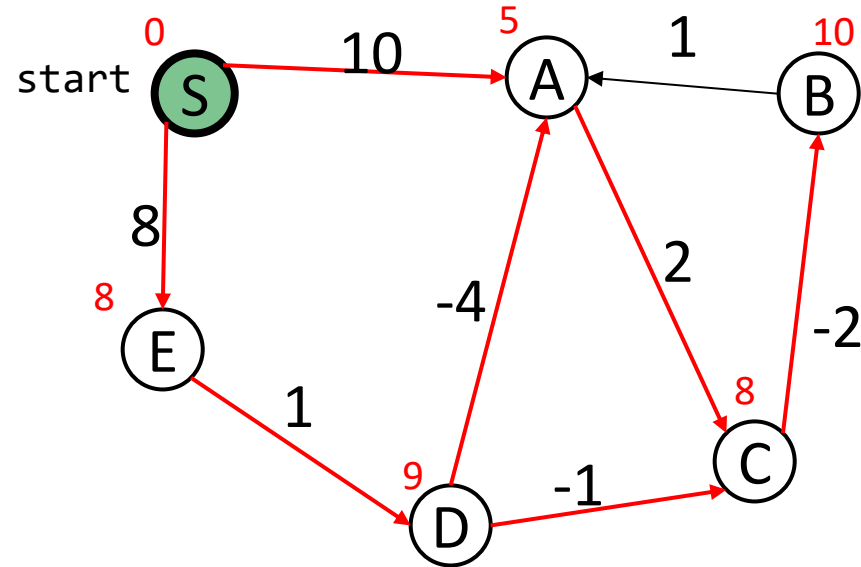
Iteration 1

node	SD	PN
S	0	-
A	10	S
B	10	C
C	12	A
D	9	E
E	8	S

* D can't use its SD of 9 to relax A and C until the *next* iteration

We assume node visit order S, A, C, B, **D**, **E** in this example.

Bellman-Ford Example

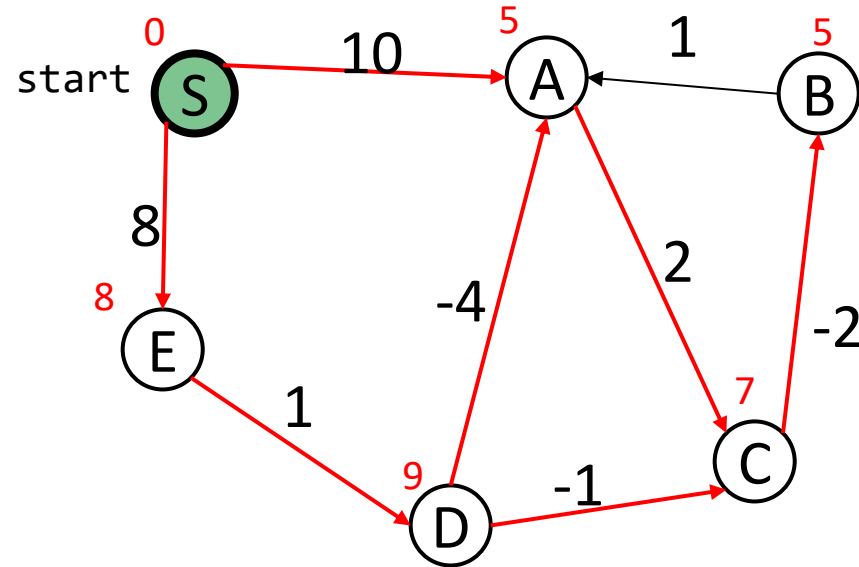


Iteration 2

node	SD	PN
S	0	-
A	10 5	S D
B	10	C
C	12 8	A D
D	9	E
E	8	S

We assume node visit order S, A, C, B, **D**, **E** in this example.

Bellman-Ford Example

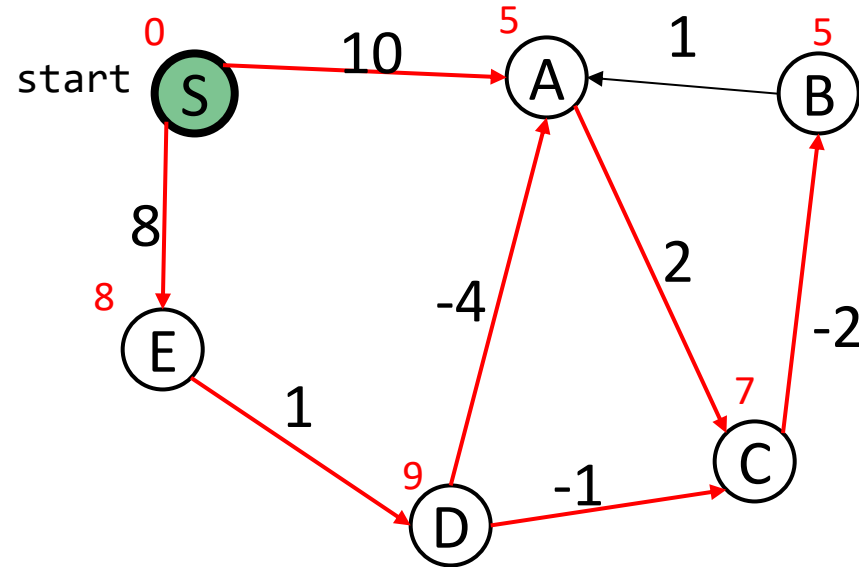


Iteration 3

node	SD	PN
S	0	-
A	5	D
B	10 5	C
C	8 7	D A
D	9	E
E	8	S

We assume node visit order S, A, C, B, **D**, **E** in this example.

Bellman-Ford Example



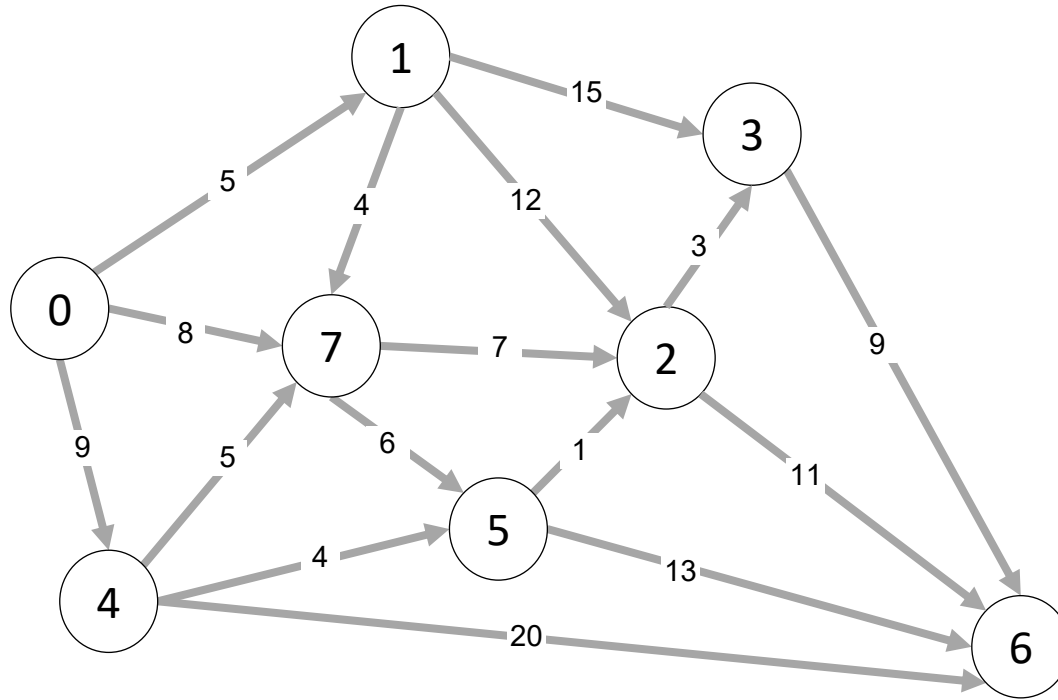
Iteration 4

node	SD	PN
S	0	-
A	5	D
B	10 5	C
C	8 7	A
D	9	E
E	8	S

No changes!
This means we can stop early

We assume node visit order S, A, C, B, **D**, **E** in this example.

Bellman-Ford Example II



v SD[]				
0	∞	0		
1	∞	5		
2	∞	17	14	
3	∞	20	17	
4	∞	9		
5	∞	13		
6	∞	28	20	25
7	∞	8		

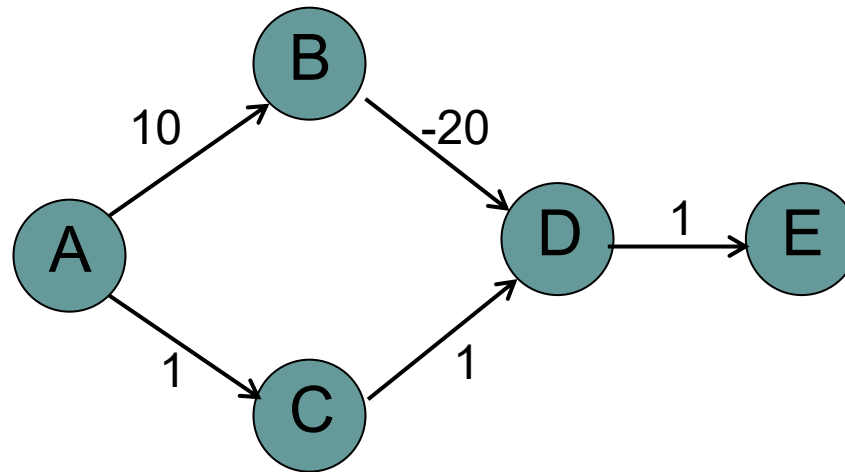
v PN[]				
0	-			
1	-	0		
2	-	1	5	
3	-	1	2	
4	-	0		
5	-	4		
6	-	2	5	2
7	-	0		

Iter 1 Iter 2 Iter 3 (converged, no further changes, so stop here)

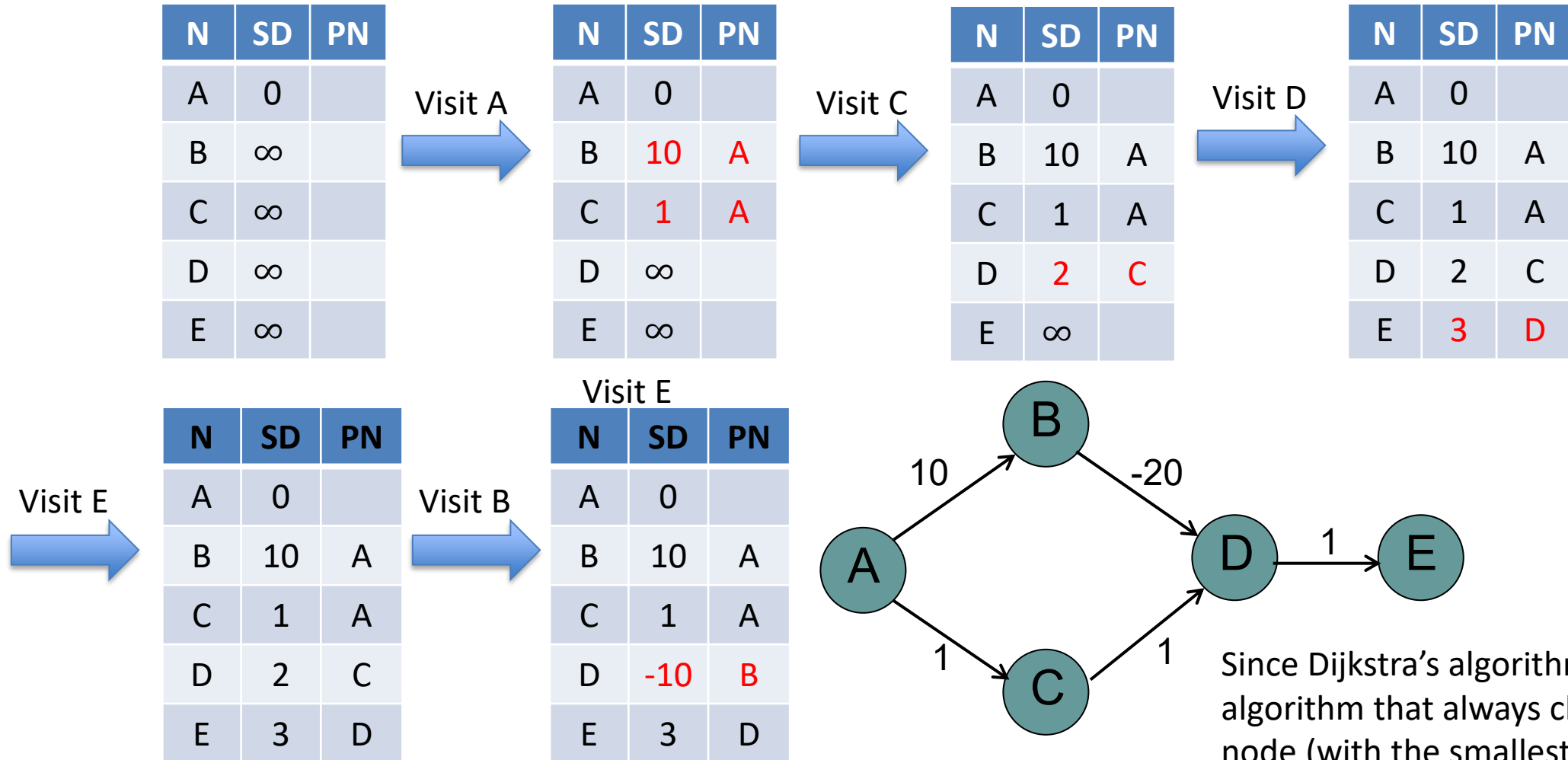
We assume node visit order 0, 1, 2, 3, 4, 5, 6, 7 in this example. Reverse order of edge relaxations will result in slower convergence, but does not affect correctness

A Toy Example with Negative Edge Weights

- Let's run Dijkstra's algorithm, Topological Sort, and Bellman Ford Algorithm on this DAG with a negative edge weight



Dijkstra's algorithm does not work for a graph w/ negative edge weights



Since Dijkstra's algorithm is a greedy algorithm that always chooses the closest unknown node (with the smallest SD), node B with SD of 10 is visited last, so node E's final SD of 3 is incorrect.

Dijkstra's algorithm does not work for a graph w/ negative edge weights

- Dijkstra's Algorithm is greedy: each node is visited once, and any node that has been visited should have its shortest distance from the source.
- After visiting A, C, D, E, we have computed D's SD is 2, but after visiting B, D's SD is updated to -10, which violates the greedy optimal assumption.
- Even if we update D's SD to be -10, its downstream node E's SD will not be updated.
- We cannot visit B before D, since we must visit the closest unknown node (with smallest SD), which is D. (Unlike topological sort, which visits B before D and gets the correct result,)

Bellman Ford works for a graph with negative edge weights

N	SD	PN
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

Iter 1

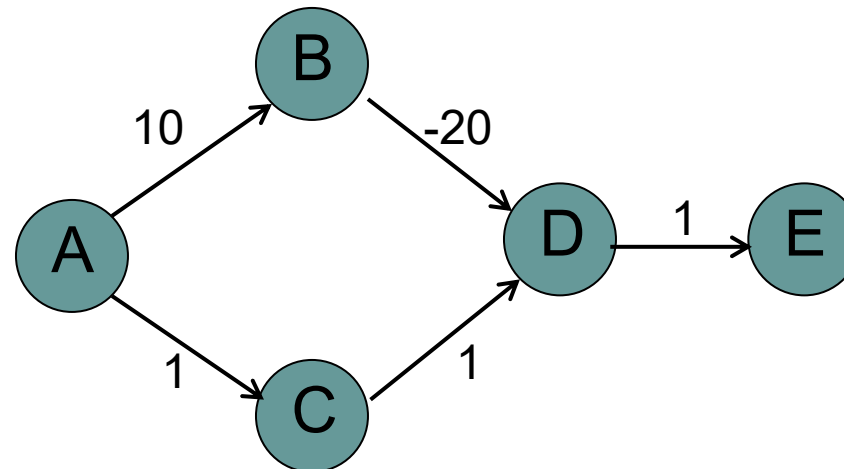
N	SD	PN
A	0	
B	10	A
C	1	A
D	-10	B
E	-9	D

Iter 2

N	SD	PN
A	0	
B	10	A
C	1	A
D	-10	B
E	-9	D

No change converged

Suppose we visit nodes in this order at each iteration: A, B, C, D, E. We run for 2 iterations (less than $V-1=3$ iterations), and converge to the correct SPT



Dijkstra's Algorithm vs. Bellman-Ford Algorithm

- Dijkstra's Algorithm:
 - Uses a priority queue to select the next node to process.
 - Greedily selects the node with the smallest tentative distance to source node.
 - Works only on graphs with non-negative edge weights.
- Bellman-Ford Algorithm:
 - Iteratively relaxes all edges $V-1$ times.
 - Can handle graphs with negative edge weights, and can detect negative cycles.
 - Relax all the edges one more time, i.e. the V -th time. Negative cycle exists if any edge can be further relaxed
 - It can find any negative cycle that is reachable from source node s (but not negative cycles that are unreachable from s).
 - If there is a negative cycle that is reachable from source node s , then any paths that go through the cycle has distance $-\infty$, since the cost can be reduced by traversing the cycle infinite number of times.
- Dijkstra's algorithm is faster and more efficient for graphs with non-negative weights; Bellman-Ford Algorithm is more versatile as it can handle negative weights and detect negative cycles, albeit at the cost of lower efficiency

Single Source Shortest-paths Algorithms Summary

Algorithm	Applicability	Worst-Case Complexity
Breadth First Search (BFS)	Unweighted, undirected or directed graph	Adjacency List: $O(V + E)$ Adjacency Matrix: $O(V^2)$
Dijkstra's algorithm	Undirected or directed graph; no negative weights/cycles	$O((V+E) \log V)$ (binary min-heap)
Bellman-Ford algorithm	Directed graph with negative weights; undirected graph with no negative weights (since a negative weight edge forms a negative cycle by itself)	$O(VE)$

Distance Vector vs. Link State Routing

Parameter	Distance Vector Routing	Link State Routing
Routing Information	Routers share their routing tables with their neighbors.	Routers share information about the entire network topology.
Convergence	Slower convergence.	Faster convergence.
Updates	Periodic updates are sent to neighboring routers.	Updates are triggered by changes in the network topology.
Algorithm	Uses the Bellman-Ford algorithm .	Uses Dijkstra's algorithm .

References

- Dijkstras Shortest Path Algorithm Explained | With Example | Graph Theory
 - <https://www.youtube.com/watch?v=bZkzH5x0SKU>
- Dijkstra's algorithm in 3 minutes
 - https://www.youtube.com/watch?v=_IHSawdgXpl
- Bellman-Ford, Michael Sambol
 - <https://www.youtube.com/watch?v=9PHkk0UavIM>
 - <https://www.youtube.com/watch?v=obWXjtg0L64>
- Bellman Ford Shortest Path Algorithm, ByteQuest
 - <https://www.youtube.com/watch?v=B5PmlJACZ9Y>
- Shortest Path Algorithms Explained (Dijkstra's & Bellman-Ford), b001
 - <https://www.youtube.com/watch?v=TtQi1LVVOUI>
- [CSE 373 WI24] Lecture 15: Shortest Path
 - https://www.youtube.com/watch?v=L8nhMwhUn4U&list=PLEcoVsAaONjd5n69K84sSmAuvTrTQT_Nl&index=14